

Hearsay source code

Sharon Basovich · UnivaBio 2026

Repository: <https://github.com/sharonbasovich/hearsay>

Base revision: 0e9618137021d0bd23b780fce12476ae42ae185c

Includes reviewed display-copy changes published on 2026-10-04.

Source snapshot SHA-256:

dc7f9f61d07c319b1301a927fa6808ab9ef65369929769e730f64d062dcf8487

Complete text of all 35 source/configuration files identified in this snapshot.

Binary WAV stimuli, screenshots and demo MP4 are not printed.

Original source files retain their exact UTF-8 contents; long lines wrap below.

AI-assisted: original code/tests/docs generated with Devin (Cognition).

This documentation update uses reviewed wording and a ReportLab rendering fallback.

.github/workflows/capture-demo.yml (45 lines)

.github/workflows/pages.yml (38 lines)

index.html (14 lines)

package.json (24 lines)

public/video/index.html (31 lines)

scripts/capture-demo.mjs (96 lines)

scripts/export-pdfs.mjs (44 lines)

scripts/make-stimuli.ts (59 lines)

scripts/render-sample.ts (33 lines)

scripts/simulate.ts (15 lines)

scripts/wav.ts (21 lines)

src/audio/hearloss.ts (46 lines)

src/audio/mix.ts (60 lines)

src/audio/player.ts (132 lines)

src/dsp/fft.ts (57 lines)

src/dsp/stimuli.ts (65 lines)

src/engine/bayes.ts (155 lines)

src/engine/family.ts (45 lines)

src/engine/headphones.ts (26 lines)

src/engine/psychometric.ts (13 lines)

src/engine/rng.ts (18 lines)

src/engine/simulate.ts (76 lines)

src/engine/staircase.ts (50 lines)

src/engine/triplets.ts (27 lines)

src/engine/types.ts (21 lines)

src/main.ts (415 lines)

src/storage.ts (34 lines)

src/style.css (114 lines)

src/ui/charts.ts (91 lines)

src/ui/icons.ts (8 lines)

tests/dsp.test.ts (108 lines)

tests/engine.test.ts (120 lines)

tests/family.test.ts (43 lines)

tsconfig.json (17 lines)

vite.config.ts (13 lines)

.github/workflows/capture-demo.yml

SHA256 231fa25e27a859cecd492b2267dd0d86f237ddc479826d2d6f493b4c59fcab40

```
1 name: Capture synthetic demo (manual)
2 on:
3   workflow_dispatch:
4 permissions:
5   contents: read
6 jobs:
7   capture:
8     if: github.repository == 'sharonbasovich/hearsay'
9     runs-on: ubuntu-latest
10    timeout-minutes: 12
11    steps:
12      - uses: actions/checkout@v4
13        with:
14          fetch-depth: 0
15      - uses: actions/setup-node@v4
16        with:
17          node-version: 22
18          cache: npm
19      - name: Record exact product snapshot
20        run: |
21          mkdir -p demo-capture
22          git rev-parse HEAD > demo-capture/source-commit.txt
23          git diff --exit-code 695c5cefe169b0b0ab5fed5fe969a4f224c60842 HEAD -- src tests index.html package.json
24          package-lock.json vite.config.ts tsconfig.json
25          git diff --name-only 695c5cefe169b0b0ab5fed5fe969a4f224c60842 HEAD > demo-capture/changed-paths.txt
26          git ls-files -z src tests index.html package.json package-lock.json vite.config.ts tsconfig.json | xargs -0
27          sha256sum > demo-capture/source-sha256.txt
28          git status --porcelain > demo-capture/source-status.txt
29      - run: npm ci
30      - run: npm test
31      - run: npm run build
32      - name: Install isolated capture tooling
33        run: |
34          npm install --no-save --package-lock=false playwright@1.58.2
35          npx playwright install --with-deps chromium
36      - name: Capture genuine synthetic UI
37        run: |
38          npm run preview -- --host 127.0.0.1 > /tmp/hearsay-preview.log 2>&1 &
39          for i in $(seq 1 30); do curl -fsS http://127.0.0.1:4173/ >/dev/null && break; sleep 1; done
40          node scripts/capture-demo.mjs
41          git diff --exit-code -- src tests index.html package.json package-lock.json vite.config.ts tsconfig.json
42      - uses: actions/upload-artifact@v4
43        if: always()
44        with:
45          name: hearsay-synthetic-demo-capture
46          path: demo-capture/
47          retention-days: 7
```

.github/workflows/pages.yml

SHA256 2ee797ab92e7d10bca2933d7108cb5aeb2259f47bdeaba542de301c155378820

```
1 name: CI and Pages
2 on:
3   push:
4     branches: [main]
5   pull_request:
6 permissions:
7   contents: read
8   pages: write
9   id-token: write
10 concurrency:
11   group: pages
12   cancel-in-progress: true
13 jobs:
14   build:
15     runs-on: ubuntu-latest
16     steps:
17       - uses: actions/checkout@v4
18       - uses: actions/setup-node@v4
19         with:
20           node-version: 22
21           cache: npm
22       - run: npm ci
23       - run: npm test
24       - run: npm run build
25       - uses: actions/upload-pages-artifact@v3
26         if: github.ref == 'refs/heads/main'
27         with:
28           path: dist
29   deploy:
30     if: github.ref == 'refs/heads/main'
31     needs: build
32     runs-on: ubuntu-latest
33     environment:
34       name: github-pages
35       url: ${ steps.deployment.outputs.page_url }
36     steps:
37       - id: deployment
38         uses: actions/deploy-pages@v4
```

index.html

SHA256 48b3ce57660accb7021328ee7028ae188a26044ecec781b3eee0042ec253ab6e

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <meta name="description" content="Hearsay: a 3-minute digits-in-noise hearing check with a Bayesian adaptive engine
and same-device family comparison. Not a diagnosis." />
7     <link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 100 100'%3E%3Ctext
y='.9em' font-size='90'%3E%F0%9F%91%82%3C/text%3E%3C/svg%3E" />
8     <title>Hearsay – a 3-minute hearing-in-noise check</title>
9   </head>
10  <body>
11    <div id="app"></div>
12    <script type="module" src="/src/main.ts"></script>
13  </body>
14 </html>
```

package.json

SHA256 3639d96add2358f7d8cb0eca5937b2603068f15eafa359beb032ec9e3ff73625

```
1 {
2   "name": "hearsay",
3   "version": "0.1.0",
4   "private": true,
5   "description": "A 3-minute digits-in-noise hearing check with a Bayesian adaptive engine, same-device family
6   comparison, and a hear-through simulator. Screening-style self-check, not a diagnosis.",
7   "type": "module",
8   "scripts": {
9     "dev": "vite",
10    "build": "tsc --noEmit && vite build && cp docs/hearsay-demo.mp4 dist/video/hearsay-demo.mp4",
11    "preview": "vite preview",
12    "test": "vitest run",
13    "typecheck": "tsc --noEmit",
14    "stimuli": "vite-node scripts/make-stimuli.ts",
15    "simulate": "vite-node scripts/simulate.ts",
16    "export:pdfs": "node scripts/export-pdfjs.mjs"
17  },
18  "devDependencies": {
19    "@types/node": "20.16.10",
20    "typescript": "5.6.3",
21    "vite": "5.4.8",
22    "vite-node": "2.1.2",
23    "vitest": "2.1.2"
24  }
```

public/video/index.html

SHA256 733a7a9a8c6231cdbadf43c458f63acc499d319de9bdf75a357316f957667fc

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <meta name="description" content="Hearsay: a 112-second synthetic demonstration of an adaptive digits-in-noise task.
Not clinically validated." />
7     <title>Hearsay: demo video</title>
8     <style>
9       body { margin: 0; background: #faf6f0; color: #1f2430; font: 16px/1.5 system-ui, sans-serif; }
10      main { max-width: 960px; margin: 0 auto; padding: 28px 18px 40px; }
11      h1 { font-size: 26px; margin: 0 0 4px; }
12      p { margin: 8px 0; color: #4a5160; }
13      .watch { display: block; padding: 24px; margin: 20px 0; border-radius: 12px; background: #1f7a74; color: white;
font-size: 24px; font-weight: 700; text-align: center; }
14      a { color: #1f7a74; }
15      .fine { font-size: 14px; }
16    </style>
17  </head>
18  <body>
19    <main>
20      <h1>Hearsay: demo video</h1>
21      <p>A 112-second demonstration of Hearsay choosing its next challenge and making uncertainty visible, made for
UnivaBio 2026. All listener responses are simulated.</p>
22      <a class="watch" href="https://www.youtube.com/watch?v=mU1zXM0ii48">Watch the current narrated demo on YouTube
(1:52)</a>
23      <p class="fine">
24        <a href="#">Try the live app</a> ·
25        <a href="https://github.com/sharonbasovich/hearsay">Source on GitHub</a>
26      </p>
27      <p class="fine">Hearsay is an educational prototype, not a medical device or hearing screening tool, and has not
been clinically validated. This demonstration assesses no person's hearing. Narration uses the stock Kokoro Heart
synthetic voice, generated offline.</p>
28    </main>
29  </body>
30 </html>
31
```

scripts/capture-demo.mjs

SHA256 0672daa0ffa9048cb32366da6ce7725c77f035d158e2e04af6593db6b10f5750

```
1 import { chromium } from 'playwright';
2 import { mkdir, writeFile } from 'node:fs/promises';
3 import assert from 'node:assert/strict';
4
5 const out = 'demo-capture';
6 await mkdir(out, { recursive: true });
7 const browser = await chromium.launch();
8 const context = await browser.newContext({
9   viewport: { width: 1920, height: 1080 },
10  recordVideo: { dir: out, size: { width: 1920, height: 1080 } },
11  locale: 'en-US', timezoneId: 'UTC',
12 });
13 // Fresh synthetic-only browser context; no microphone/camera permissions,
14 // credentials, network mocks, injected answers, or product DOM/state changes.
15 const page = await context.newPage();
16 const errors = [];
17 page.on('pageerror', error => errors.push({ type: 'pageerror', message: error.message }));
18 page.on('requestfailed', request => errors.push({ type: 'requestfailed', url: request.url(), failure: request.failure()
19 }));
20 const started = performance.now();
21 const marks = [];
22 let status = { status: 'running' };
23 const mark = async (name, hold = 0) => {
24   marks.push({ name, seconds: (performance.now() - started) / 1000 });
25   await page.screenshot({ path: `${out}/${name}.png` });
26   if (hold) await page.waitForTimeout(hold * 1000);
27 };
28 const body = () => page.locator('#app').innerText();
29 try {
30   await page.goto('http://127.0.0.1:4173/', { waitUntil: 'networkidle' });
31   assert.match(await body(), /educational prototype has not been clinically validated/i);
32   await mark('01-home', 8);
33   await page.getByRole('link', { name: 'Lab', exact: true }).click();
34   await page.getByRole('link', { name: 'Watch a virtual listener take the check', exact: true }).waitFor();
35   await mark('02-lab-entry', 6);
36   await page.getByRole('link', { name: 'Watch a virtual listener take the check', exact: true }).click();
37   await page.locator('#name').waitFor();
38   assert.match(await body(), /Results are simulated/);
39   assert.equal(await page.locator('#name').inputValue(), 'Virtual listener');
40   assert.equal(await page.locator('#device').inputValue(), 'Simulated');
41   await mark('03-virtual-setup', 6);
42   await page.getByRole('button', { name: 'Continue', exact: true }).click();
43   await page.locator('.keypad').waitFor();
44   assert.match(await body(), /Synthetic demo: responses and this result are simulated/);
45   await mark('04-virtual-start');
46   const seen = new Set();
47   const deadline = Date.now() + 150_000;
48   while (true) {
49     const current = await body();
50     if (!current.includes('Type the three digits you heard')) break;
51     assert.ok(Date.now() < deadline, 'Virtual run did not finish within 150 seconds');
52     const trial = current.match(/triplet\s+(\d+)/i)?.[0] || '';
53     assert.ok(trial, 'Active trial label missing');
54     if (!seen.has(trial)) {
55       seen.add(trial);
56       const number = trial.match(/triplet\s+(\d+)/i)?.[1];
57       await mark(`trial-${String(number).padStart(2, '0')}`);
58     }
59     await page.waitForTimeout(120);
60   }
61   assert.match(await body(), /model-estimated task threshold/);
62   assert.match(await body(), /Synthetic demo: responses and this result are simulated/);
63   await mark('05-virtual-result', 18);
64   await writeFile(`${out}/virtual-result.txt`, await body());
65   // No result is saved to a family board. Only the built-in simulation follows.
66   await page.getByRole('link', { name: 'Lab', exact: true }).click();
67   await page.locator('#n').waitFor();
68   assert.equal(await page.locator('#n').inputValue(), '400');
69   assert.equal(await page.locator('#slope').inputValue(), '0.7');
70   await mark('06-lab-settings', 6);
71   await page.getByRole('button', { name: 'Run simulation', exact: true }).click();
72   await page.locator('table.results tbody tr').nth(2).waitFor();
73   const rows = await page.locator('table.results tbody tr').allTextContents();
74   assert.match(rows[0], /\0.79/);
75   assert.match(rows[1], /\1.53/);
76   assert.match(rows[2], /\11\8/);
77   assert.match(rows[2], /\1\01/);
78   await mark('07-lab-results', 32);
```

scripts/capture-demo.mjs (continued)

```
78   await writeFile(`${out}/lab-result.txt`, await body());
79   await page.getByRole('link', { name: 'Honesty', exact: true }).click();
80   await page.getByRole('heading', { name: "What's real, what's synthetic, what's missing", exact: true }).waitFor();
81   await mark('08-honesty', 5);
82   assert.deepEqual(errors, [], 'Unexpected browser errors');
83   await writeFile(`${out}/ui-verification.txt`, 'Verified corrected copy; built-in virtual-listener entry; automatic
genuine response/posterior/track updates; synthetic result; default400/slope0.7 simulation and expected RMSE cells. No
human responses, saved result, permissions or product state injection. App audio must be omitted from the narrated final
edit.\n');
84   status = { status: 'passed' };
85 } catch (error) {
86   status = { status: 'failed', error: String(error), stack: error?.stack || null };
87   throw error;
88 } finally {
89   await writeFile(`${out}/capture-status.json`, JSON.stringify(status, null, 2));
90   await writeFile(`${out}/browser-errors.json`, JSON.stringify(errors, null, 2));
91   await writeFile(`${out}/marks.json`, JSON.stringify({ sourceCommit: process.env.GITHUB_SHA || null, viewport: {
width:1920, height:1080 }, marks }, null, 2));
92   const video = page.video();
93   await context.close();
94   if (video) await video.saveAs(`${out}/hearsay-current-build.webm`);
95   await browser.close();
96 }
```


scripts/export-pdfs.mjs

SHA256 882a02db564748428c569bdf20c6e76dcc25a2be2ab3a5d4779face68198e0e6

```
1 // Regenerates docs/hearsay-one-pager.pdf and docs/hearsay-source-code.pdf with headless Chrome.
2 // Usage: node scripts/export-pdfs.mjs (set CHROME=/path/to/chrome if not on PATH as google-chrome)
3 import { execFileSync } from "node:child_process";
4 import { mkdtempSync, readFileSync, writeFileSync } from "node:fs";
5 import { tmpdir } from "node:os";
6 import { join, resolve } from "node:path";
7
8 const root = resolve(import.meta.dirname, "..");
9 const chrome = process.env.CHROME ?? "google-chrome";
10
11 function printPdf(htmlPath, outputPath) {
12   execFileSync(chrome, [
13     "--headless=new",
14     "--disable-gpu",
15     "--no-pdf-header-footer",
16     "--allow-file-access-from-files",
17     `--print-to-pdf=${outputPath}`,
18     `file://${htmlPath}`,
19   ], { stdio: "ignore" });
20   console.log("wrote", outputPath);
21 }
22
23 const esc = (s) => s.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;");
24 const sources = execFileSync("git", [
25   "ls-files", "src", "scripts", "tests", "index.html", "package.json", "vite.config.ts", "tsconfig.json",
26   ".github/workflows/pages.yml", "public/video/index.html",
27 ], { cwd: root, encoding: "utf8" }).split("\n").filter(Boolean);
28 const text = Object.fromEntries(sources.map((f) => [f, readFileSync(join(root, f), "utf8")]));
29 const toc = sources.map((f) => `- ${f} (${text[f].split("\n").length} lines)</li>`).join("");
30 const body = sources.map((f) => `

## ${f}</h2><pre>${esc(text[f])}</pre>`).join(""); 31 const codeHtml = `<!doctype html><html><head><meta charset="utf-8"><style> 32 @page{size:Letter;margin:.5in}body{font:10px "DejaVu Sans",sans-serif}h1{font-size:20px} 33 h2{font-size:12px;background:#eee;padding:3px;page-break-after:avoid} 34 pre{font:7.5px/1.25 "DejaVu Sans Mono",monospace;white-space:pre-wrap;word-break:break-all} 35 </style></head><body> 36 <h1>Hearsay: source code</h1> 37 <p>Sharon Basovich, UnivaBio 2026. Repository: https://github.com/sharonbasovich/hearsay · Live: 38 https://sharonbasovich.github.io/hearsay/</p> 39 <p>AI-assisted: code, tests and docs were generated with the Devin AI coding agent (Cognition). Binary assets are 40 omitted: WAV stimuli in public/stimuli/ (ElevenLabs TTS, generated offline by scripts/make-stimuli.ts) and the demo 41 MP4.</p> 42 <ol>${toc}</ol>${body}</body></html>`; 43 const tmp = mkdtempSync(join(tmpdir(), "hearsay-pdf-")); 44 const codePath = join(tmp, "source-code.html"); 45 writeFileSync(codePath, codeHtml); 46 47 printPdf(join(root, "docs/one-pager.html"), join(root, "docs/hearsay-one-pager.pdf")); 48 printPdf(codePath, join(root, "docs/hearsay-source-code.pdf"));

```

scripts/make-stimuli.ts

SHA256 7a0971b1006752940d6f26f488cf650c450fdb7465082f9e190bc0b67b11593b

```
1 /**
2  * Generates the digit tokens, demo sentence and speech-shaped noise in public/stimuli.
3  * Speech is synthesized with the ElevenLabs text-to-speech API (requires ELEVENLABS_API_KEY);
4  * the noise is derived locally from the digit tokens' long-term spectrum.
5  */
6 import { mkdirSync, writeFileSync } from "node:fs";
7 import { join } from "node:path";
8 import { normalizeRms, speechShapedNoise, trimSilence } from "../src/dsp/stimuli";
9 import { encodeWav } from "../wav";
10
11 const SR = 24000;
12 const VOICE_ID = process.env.HEARSAY_VOICE_ID ?? "21m00Tcm4TlvDq8ikWAM";
13 const MODEL = "eleven_multilingual_v2";
14 const OUT = join(process.cwd(), "public", "stimuli");
15 export const DIGIT_WORDS: Record<number, string> = { 0: "zero", 1: "one", 2: "two", 3: "three", 4: "four", 5: "five", 6:
16   "six", 8: "eight", 9: "nine" };
17 const SENTENCE =
18   "Hi Grandpa! I'm picking you up at four fifteen on Thursday. Bring your blue jacket, and don't forget the pharmacy slip
19   for Doctor Patel.";
20
21 async function tts(text: string): Promise<Float32Array> {
22   const key = process.env.ELEVENLABS_API_KEY;
23   if (!key) throw new Error("ELEVENLABS_API_KEY is not set");
24   const res = await fetch(`https://api.elevenlabs.io/v1/text-to-speech/${VOICE_ID}?output_format=pcm_24000`, {
25     method: "POST",
26     headers: { "xi-api-key": key, "content-type": "application/json" },
27     body: JSON.stringify({ text, model_id: MODEL, voice_settings: { stability: 0.8, similarity_boost: 0.7 } }),
28   });
29   if (!res.ok) throw new Error(`TTS failed for "${text}": ${res.status} ${await res.text()}`);
30   const buf = Buffer.from(await res.arrayBuffer());
31   const out = new Float32Array(buf.length / 2);
32   for (let i = 0; i < out.length; i++) out[i] = buf.readInt16LE(i * 2) / 32768;
33   return out;
34 }
35
36 async function main(): Promise<void> {
37   mkdirSync(OUT, { recursive: true });
38   const tokens: Float32Array[] = [];
39   const manifest: Record<string, number> = {};
40   for (const [digit, word] of Object.entries(DIGIT_WORDS)) {
41     const raw = await tts(`${word}.`);
42     const tok = normalizeRms(trimSilence(raw, SR));
43     tokens.push(tok);
44     manifest[digit] = tok.length / SR;
45     writeFileSync(join(OUT, `digit-${digit}.wav`), encodeWav(tok, SR));
46     console.log(`digit ${digit}: ${((tok.length / SR).toFixed(3))} s`);
47   }
48   const noise = speechShapedNoise(tokens, 1 << 17, 20261006);
49   writeFileSync(join(OUT, "noise.wav"), encodeWav(noise, SR));
50   const sentence = normalizeRms(trimSilence(await tts(SENTENCE), SR, -45, 60));
51   writeFileSync(join(OUT, "sentence.wav"), encodeWav(sentence, SR));
52   writeFileSync(
53     join(OUT, "manifest.json"),
54     JSON.stringify({ sampleRate: SR, voiceId: VOICE_ID, model: MODEL, generatedAt: new Date().toISOString(),
55       digitDurations: manifest, sentence: SENTENCE }, null, 2) + "\n",
56   );
57 }
58
59 main().catch((e) => {
60   console.error(e);
61   process.exit(1);
62 });
```

scripts/render-sample.ts

SHA256 3ad133772770c5d6e17d4d1ef1162a3f68de5dc81cad42f0453bd88828d7f6c

```
1  /** Renders a sample antiphasic triplet to a stereo WAV (used for the demo video). Usage: vite-node scripts/render-
2  sample.ts out.wav 5 2 8 -2 */
3  import { readFileSync, writeFileSync } from "node:fs";
4  import { mixTriplet } from "../src/audio/mix";
5
6  function readWav(path: string): Float32Array {
7    const b = readFileSync(path);
8    const n = b.readUInt32LE(40) / 2;
9    const out = new Float32Array(n);
10   for (let i = 0; i < n; i++) out[i] = b.readInt16LE(44 + i * 2) / 32768;
11   return out;
12 }
13
14 const [out, a, b, c, snr] = process.argv.slice(2);
15 const sr = 24000;
16 const m = mixTriplet([a, b, c].map((d) => readWav(`public/stimuli/digit-${d}.wav`)), readWav("public/stimuli/noise.wav"),
17   { sampleRate: sr, snr: Number(snr), antiphasic: true });
18 const buf = Buffer.alloc(44 + m.left.length * 4);
19 buf.write("RIFF", 0);
20 buf.writeUInt32LE(36 + m.left.length * 4, 4);
21 buf.write("WAVEfmt ", 8);
22 buf.writeUInt32LE(16, 16);
23 buf.writeUInt16LE(1, 20);
24 buf.writeUInt16LE(2, 22);
25 buf.writeUInt32LE(sr, 24);
26 buf.writeUInt32LE(sr * 4, 28);
27 buf.writeUInt16LE(4, 32);
28 buf.writeUInt16LE(16, 34);
29 buf.write("data", 36);
30 buf.writeUInt32LE(m.left.length * 4, 40);
31 for (let i = 0; i < m.left.length; i++) {
32   buf.writeInt16LE(Math.round(Math.max(-1, Math.min(1, m.left[i])) * 32767), 44 + i * 4);
33   buf.writeInt16LE(Math.round(Math.max(-1, Math.min(1, m.right[i])) * 32767), 46 + i * 4);
34 }
35 writeFileSync(out, buf);
```

scripts/simulate.ts

SHA256 9ba1dee3c7e7bbe2ce5df2108c09ff716fdeaac68c8fed1f1d2b853cd8ba941f

```
1 import { writeFileSync } from "node:fs";
2 import { DEFAULT_SIM, METHODS, simulate } from "../src/engine/simulate";
3
4 const slopes = [0.5, 0.7, 1.0];
5 const rows: Record<string, unknown>[] = [];
6 for (const trueSlope of slopes) {
7   for (const [key, make] of Object.entries(METHODS)) {
8     const s = simulate(make, { ...DEFAULT_SIM, trueSlope });
9     rows.push({ key, method: s.method, trueSlope, rmse: s.rmse, bias: s.bias, meanTrials: s.meanTrials, coverage90:
10       s.coverage90, retestSd: s.retestSd });
11     console.log(
12       `${s.method.padEnd(22)} slope=${trueSlope} RMSE=${s.rmse.toFixed(2)} dB bias=${s.bias.toFixed(2)}
13       trials=${s.meanTrials.toFixed(1)} cover90=${(s.coverage90 * 100).toFixed(0)}% retestSD=${s.retestSd.toFixed(2)}`,
14     );
15   }
16 }
17 writeFileSync("docs/validation.json", JSON.stringify({ config: DEFAULT_SIM, slopes, rows }, null, 2) + "\n");
```

SHA256 66ebfe33de3fb1215221514cea0b3669870edc4a6bc8d7b91137996ce1f63960

```
1 export function encodeWav(x: Float32Array, sampleRate: number): Buffer {
2   const buf = Buffer.alloc(44 + x.length * 2);
3   buf.write("RIFF", 0);
4   buf.writeUInt32LE(36 + x.length * 2, 4);
5   buf.write("WAVE", 8);
6   buf.write("fmt ", 12);
7   buf.writeUInt32LE(16, 16);
8   buf.writeUInt16LE(1, 20);
9   buf.writeUInt16LE(1, 22);
10  buf.writeUInt32LE(sampleRate, 24);
11  buf.writeUInt32LE(sampleRate * 2, 28);
12  buf.writeUInt16LE(2, 32);
13  buf.writeUInt16LE(16, 34);
14  buf.write("data", 36);
15  buf.writeUInt32LE(x.length * 2, 40);
16  for (let i = 0; i < x.length; i++) {
17    const v = Math.max(-1, Math.min(1, x[i]));
18    buf.writeInt16LE(Math.round(v * 32767), 44 + i * 2);
19  }
20  return buf;
21 }
```

src/audio/hearloss.ts

SHA256 f5e3f960bf0b804b56ef3e154e420691a21b0a51676d577cd2d812d104ddfe30

```
1 /** Audiogram frequencies (Hz) used by the hear-through simulator. */
2 export const AUDIOGRAM_FREQS = [250, 500, 1000, 2000, 4000, 8000] as const;
3
4 export interface Profile {
5   id: string;
6   label: string;
7   description: string;
8   /** Attenuation in dB at each AUDIOGRAM_FREQS entry. Illustrative only. */
9   loss: number[];
10 }
11
12 export const PROFILES: Profile[] = [
13   { id: "none", label: "Typical", description: "No filtering.", loss: [0, 0, 0, 0, 0, 0] },
14   {
15     id: "mild-hf",
16     label: "Mild high-frequency",
17     description: "Illustrative gentle slope above 2 kHz – consonants like s, f and th start to blur.",
18     loss: [0, 0, 5, 20, 35, 45],
19   },
20   {
21     id: "moderate-sloping",
22     label: "Moderate sloping",
23     description: "Illustrative steeper slope – vowels stay loud, so speech sounds 'there' but unclear.",
24     loss: [10, 15, 30, 45, 60, 70],
25   },
26   {
27     id: "noise-notch",
28     label: "Noise notch",
29     description: "Illustrative dip around 4 kHz, a pattern often discussed with long-term loud-noise exposure.",
30     loss: [0, 0, 5, 15, 40, 20],
31   },
32 ];
33
34 /** Linear interpolation (in log-frequency) of a profile's attenuation at frequency f. */
35 export function lossAt(profile: Profile, f: number): number {
36   const fs = AUDIOGRAM_FREQS;
37   if (f <= fs[0]) return profile.loss[0];
38   if (f >= fs[fs.length - 1]) return profile.loss[fs.length - 1];
39   for (let i = 0; i < fs.length - 1; i++) {
40     if (f >= fs[i] && f <= fs[i + 1]) {
41       const t = Math.log(f / fs[i]) / Math.log(fs[i + 1] / fs[i]);
42       return profile.loss[i] + t * (profile.loss[i + 1] - profile.loss[i]);
43     }
44   }
45   return 0;
46 }
```

SHA256 10e6ad33e39e9dde819b91228628f2566fe1e6f0a9bd0d3ac54e929ddaead7b1

```

1 import { dbToGain } from "../dsp/fft";
2
3 export interface MixOptions {
4   sampleRate: number;
5   snr: number;
6   /** Invert the speech in the right ear (antiphasic presentation); noise stays identical in both ears. */
7   antiphasic: boolean;
8   noiseGain?: number;
9   leadMs?: number;
10  gapMs?: number;
11  tailMs?: number;
12  noiseOffset?: number;
13 }
14
15 export interface StereoMix {
16   left: Float32Array;
17   right: Float32Array;
18   onsets: number[];
19 }
20
21 /** Mix three digit tokens into looping speech-shaped noise at the requested SNR. */
22 export function mixTriplet(tokens: Float32Array[], noise: Float32Array, o: MixOptions): StereoMix {
23   const sr = o.sampleRate;
24   const noiseGain = o.noiseGain ?? 0.35;
25   const lead = Math.round(((o.leadMs ?? 500) / 1000) * sr);
26   const gap = Math.round(((o.gapMs ?? 300) / 1000) * sr);
27   const tail = Math.round(((o.tailMs ?? 400) / 1000) * sr);
28   const len = lead + tokens.reduce((a, t) => a + t.length, 0) + gap * (tokens.length - 1) + tail;
29   const left = new Float32Array(len);
30   const right = new Float32Array(len);
31   const off = o.noiseOffset ?? 0;
32   const ramp = Math.round(0.02 * sr);
33   for (let i = 0; i < len; i++) {
34     const env = Math.min(1, i / ramp, (len - 1 - i) / ramp);
35     const n = noise[(i + off) % noise.length] * noiseGain * env;
36     left[i] = n;
37     right[i] = n;
38   }
39   const sg = noiseGain * dbToGain(o.snr);
40   const onsets: number[] = [];
41   let pos = lead;
42   for (const t of tokens) {
43     onsets.push(pos / sr);
44     for (let i = 0; i < t.length; i++) {
45       left[pos + i] += t[i] * sg;
46       right[pos + i] += (o.antiphasic ? -1 : 1) * t[i] * sg;
47     }
48     pos += t.length + gap;
49   }
50   let peak = 0;
51   for (let i = 0; i < len; i++) peak = Math.max(peak, Math.abs(left[i]), Math.abs(right[i]));
52   if (peak > 0.95) {
53     const g = 0.95 / peak;
54     for (let i = 0; i < len; i++) {
55       left[i] *= g;
56       right[i] *= g;
57     }
58   }
59   return { left, right, onsets };
60 }

```

src/audio/player.ts

SHA256 34b8bde09568140581f10324d758dba1b1a13cdc106b486af0061d7cdd38d542

```
1 import { mixTriplet } from "../mix";
2 import { AUDIOGRAM_FREQS, type Profile } from "../hearloss";
3 import type { Digit } from "../engine/triplets";
4
5 export class AudioEngine {
6   readonly ctx: AudioContext;
7   readonly master: GainNode;
8   private tokens = new Map<number, Float32Array>();
9   private noise = new Float32Array(0);
10  private sentence: AudioBuffer | null = null;
11  private current: AudioBufferSourceNode | null = null;
12  private loaded: Promise<void> | null = null;
13  private noiseOffset = 0;
14
15  constructor() {
16    this.ctx = new AudioContext();
17    this.master = this.ctx.createGain();
18    this.master.gain.value = 0.5;
19    this.master.connect(this.ctx.destination);
20  }
21
22  load(base = "../stimuli/"): Promise<void> {
23    this.loaded ??= (async () => {
24      const get = async (name: string) => {
25        const res = await fetch(base + name);
26        if (!res.ok) throw new Error(`Could not load ${name}`);
27        return this.ctx.decodeAudioData(await res.arrayBuffer());
28      };
29      const digits: Digit[] = [0, 1, 2, 3, 4, 5, 6, 8, 9];
30      await Promise.all(digits.map(async (d) => this.tokens.set(d, (await get(`digit-${d}.wav`)).getChannelData(0))));
31      this.noise = (await get("noise.wav")).getChannelData(0);
32      this.sentence = await get("sentence.wav");
33    })();
34    return this.loaded;
35  }
36
37  setVolume(v: number): void {
38    this.master.gain.setTargetAtTime(v, this.ctx.currentTime, 0.02);
39  }
40
41  stop(): void {
42    try {
43      this.current?.stop();
44    } catch {
45      /* already stopped */
46    }
47    this.current = null;
48  }
49
50  private play(buf: AudioBuffer, dest: AudioNode = this.master): Promise<void> {
51    this.stop();
52    const src = this.ctx.createBufferSource();
53    src.buffer = buf;
54    src.connect(dest);
55    this.current = src;
56    return new Promise((resolve) => {
57      src.onended = () => resolve();
58      src.start();
59    });
60  }
61
62  async playTriplet(digits: readonly Digit[], snr: number, antiphasic = true, dest?: AudioNode): Promise<void> {
63    await this.ctx.resume();
64    const mix = mixTriplet(
65      digits.map((d) => this.tokens.get(d!)),
66      this.noise,
67      { sampleRate: this.ctx.sampleRate, snr, antiphasic, noiseOffset: this.noiseOffset },
68    );
69    this.noiseOffset = (this.noiseOffset + 9973) % Math.max(1, this.noise.length);
70    const buf = this.ctx.createBuffer(2, mix.left.length, this.ctx.sampleRate);
71    buf.copyToChannel(mix.left, 0);
72    buf.copyToChannel(mix.right, 1);
73    return this.play(buf, dest);
74  }
75
76  /** Three 200 Hz tones; `quiet` is 6 dB softer, `antiphase` is phase-inverted in the right ear. */
77  async playHeadphoneTrial(quiet: number, antiphase: number): Promise<void> {
78    await this.ctx.resume();
```


src/audio/player.ts (continued)

```
79     const sr = this.ctx.sampleRate;
80     const toneLen = Math.round(sr * 0.8);
81     const gapLen = Math.round(sr * 0.5);
82     const len = toneLen * 3 + gapLen * 2;
83     const buf = this.ctx.createBuffer(2, len, sr);
84     const l = buf.getChannelData(0);
85     const r = buf.getChannelData(1);
86     for (let t = 0; t < 3; t++) {
87         const amp = 0.3 * (t === quiet ? 0.5 : 1);
88         const start = t * (toneLen + gapLen);
89         for (let i = 0; i < toneLen; i++) {
90             const env = Math.min(1, i / (0.05 * sr), (toneLen - i) / (0.05 * sr));
91             const v = amp * env * Math.sin((2 * Math.PI * 200 * i) / sr);
92             l[start + i] = v;
93             r[start + i] = t === antiphase ? -v : v;
94         }
95     }
96     return this.play(buf);
97 }
98
99 /** Chain of peaking filters approximating an illustrative attenuation profile. */
100 lossChain(profile: Profile): AudioNode {
101     let head: AudioNode = this.master;
102     const nodes = [...AUDIOGRAM_FREQS].reverse().map((f, idx) => {
103         const i = AUDIOGRAM_FREQS.length - 1 - idx;
104         const bq = this.ctx.createBiquadFilter();
105         if (i === AUDIOGRAM_FREQS.length - 1) {
106             bq.type = "highshelf";
107             bq.frequency.value = f * 0.75;
108         } else {
109             bq.type = "peaking";
110             bq.frequency.value = f;
111             bq.Q.value = 1.0;
112         }
113         bq.gain.value = -profile.loss[i];
114         return bq;
115     });
116     for (const n of nodes) {
117         n.connect(head);
118         head = n;
119     }
120     return head;
121 }
122
123 async playSentence(profile: Profile | null): Promise<void> {
124     await this.ctx.resume();
125     if (!this.sentence) return;
126     return this.play(this.sentence, profile ? this.lossChain(profile) : this.master);
127 }
128
129 async playTripletThrough(profile: Profile, digits: readonly Digit[], snr: number): Promise<void> {
130     return this.playTriplet(digits, snr, false, this.lossChain(profile));
131 }
132 }
```

SHA256 d72e8193b4a489aa1b0b535a8d4232af3b305af24213c670ec81e1eded4f3506

```

1 export function fft(re: Float64Array, im: Float64Array, inverse = false): void {
2   const n = re.length;
3   if (n !== im.length || (n & (n - 1)) !== 0) throw new Error("FFT length must be a power of two");
4   for (let i = 1, j = 0; i < n; i++) {
5     let bit = n >> 1;
6     for (; j & bit; bit >>= 1) j ^= bit;
7     j ^= bit;
8     if (i < j) {
9       [re[i], re[j]] = [re[j], re[i]];
10      [im[i], im[j]] = [im[j], im[i]];
11    }
12  }
13  for (let len = 2; len <= n; len <= 1) {
14    const ang = ((inverse ? 2 : -2) * Math.PI) / len;
15    const wr = Math.cos(ang);
16    const wi = Math.sin(ang);
17    for (let i = 0; i < n; i += len) {
18      let cr = 1;
19      let ci = 0;
20      for (let k = 0; k < len / 2; k++) {
21        const a = i + k;
22        const b = a + len / 2;
23        const tr = re[b] * cr - im[b] * ci;
24        const ti = re[b] * ci + im[b] * cr;
25        re[b] = re[a] - tr;
26        im[b] = im[a] - ti;
27        re[a] += tr;
28        im[a] += ti;
29        const ncr = cr * wr - ci * wi;
30        ci = cr * wi + ci * wr;
31        cr = ncr;
32      }
33    }
34  }
35  if (inverse) {
36    for (let i = 0; i < n; i++) {
37      re[i] /= n;
38      im[i] /= n;
39    }
40  }
41 }
42
43 export function rms(x: ArrayLike<number>): number {
44   let s = 0;
45   for (let i = 0; i < x.length; i++) s += x[i] * x[i];
46   return Math.sqrt(s / Math.max(1, x.length));
47 }
48
49 export function dbToGain(db: number): number {
50   return Math.pow(10, db / 20);
51 }
52
53 export function nextPow2(n: number): number {
54   let p = 1;
55   while (p < n) p <= 1;
56   return p;
57 }

```

src/dsp/stimuli.ts

SHA256 243a58598d29fe40234f7bf8ec02bedd3a90289b2aeee465f2e68c09d4d6df0e

```
1 import { fft, nextPow2, rms } from "./fft";
2 import { gaussian, mulberry32 } from "../engine/rng";
3
4 export const TARGET_RMS = 0.1;
5
6 export function trimSilence(x: Float32Array, sampleRate: number, thresholdDb = -40, padMs = 15): Float32Array {
7   let peak = 0;
8   for (let i = 0; i < x.length; i++) peak = Math.max(peak, Math.abs(x[i]));
9   if (peak === 0) return x.slice(0, 0);
10  const thr = peak * Math.pow(10, thresholdDb / 20);
11  let start = 0;
12  while (start < x.length && Math.abs(x[start]) < thr) start++;
13  let end = x.length - 1;
14  while (end > start && Math.abs(x[end]) < thr) end--;
15  const pad = Math.round((padMs / 1000) * sampleRate);
16  return x.slice(Math.max(0, start - pad), Math.min(x.length, end + pad + 1));
17 }
18
19 export function normalizeRms(x: Float32Array, target = TARGET_RMS): Float32Array {
20   const r = rms(x);
21   if (r === 0) return x.slice();
22   const g = target / r;
23   const out = new Float32Array(x.length);
24   for (let i = 0; i < x.length; i++) out[i] = x[i] * g;
25   return out;
26 }
27
28 export function averagePowerSpectrum(signals: Float32Array[], n: number): Float64Array {
29   const power = new Float64Array(n);
30   for (const s of signals) {
31     const re = new Float64Array(n);
32     const im = new Float64Array(n);
33     for (let i = 0; i < Math.min(n, s.length); i++) re[i] = s[i];
34     fft(re, im);
35     for (let k = 0; k < n; k++) power[k] += re[k] * re[k] + im[k] * im[k];
36   }
37   for (let k = 0; k < n; k++) power[k] /= Math.max(1, signals.length);
38   return power;
39 }
40
41 /** Stationary noise whose long-term spectrum matches the speech tokens; circular so it loops seamlessly. */
42 export function speechShapedNoise(signals: Float32Array[], length: number, seed: number, target = TARGET_RMS):
  Float32Array {
43   const n = nextPow2(length);
44   const specN = nextPow2(Math.max(...signals.map((s) => s.length)));
45   const p = averagePowerSpectrum(signals, specN);
46   const rng = mulberry32(seed);
47   const re = new Float64Array(n);
48   const im = new Float64Array(n);
49   for (let k = 0; k <= n / 2; k++) {
50     const src = Math.min(specN - 1, Math.round((k / n) * specN));
51     const mag = Math.sqrt(p[src]);
52     const a = gaussian(rng);
53     const b = gaussian(rng);
54     re[k] = mag * a;
55     im[k] = k === 0 || k === n / 2 ? 0 : mag * b;
56     if (k > 0 && k < n / 2) {
57       re[n - k] = re[k];
58       im[n - k] = -im[k];
59     }
60   }
61   fft(re, im, true);
62   const out = new Float32Array(n);
63   for (let i = 0; i < n; i++) out[i] = re[i];
64   return normalizeRms(out, target);
65 }
```

src/engine/bayes.ts

SHA256 ac7b19e82b48f4c3a5c3593317789d5d49b66ad7518adfd492c9735061407116

```
1 import { DEFAULT_LAPSE, pCorrect } from "../psychometric";
2 import type { AdaptiveProcedure, Estimate, Trial } from "../types";
3
4 export interface BayesOptions {
5   srtMin?: number;
6   srtMax?: number;
7   srtStep?: number;
8   slopes?: number[];
9   priorMean?: number;
10  priorSd?: number;
11  lapse?: number;
12  candidates?: number[];
13  minTrials?: number;
14  maxTrials?: number;
15  targetSd?: number;
16 }
17
18 function range(a: number, b: number, step: number): number[] {
19   const out: number[] = [];
20   for (let x = a; x <= b + 1e-9; x += step) out.push(Math.round(x * 1000) / 1000);
21   return out;
22 }
23
24 /**
25  * Grid-based Bayesian adaptive procedure (in the spirit of QUEST+/psi).
26  * Joint posterior over SRT and psychometric slope; each trial is placed at the SNR that
27  * minimizes the expected posterior entropy of the SRT, and the run stops once the
28  * SRT posterior is tight enough.
29  */
30 export class BayesianDin implements AdaptiveProcedure {
31   readonly name = "Hearsay Bayesian";
32   readonly srts: number[];
33   readonly slopes: number[];
34   readonly candidates: number[];
35   private post: Float64Array;
36   private readonly o: Required<BayesOptions>;
37   private readonly lik: Float64Array[];
38   private n = 0;
39
40   constructor(opts: BayesOptions = {}) {
41     this.o = {
42       srtMin: -22,
43       srtMax: 10,
44       srtStep: 0.25,
45       slopes: [0.4, 0.7, 1.1],
46       priorMean: -8,
47       priorSd: 7,
48       lapse: DEFAULT_LAPSE,
49       candidates: range(-22, 10, 1),
50       minTrials: 10,
51       maxTrials: 24,
52       targetSd: 1.25,
53       ...opts,
54     };
55     this.srts = range(this.o.srtMin, this.o.srtMax, this.o.srtStep);
56     this.slopes = this.o.slopes;
57     this.candidates = this.o.candidates;
58     const S = this.srts.length;
59     const K = this.slopes.length;
60     this.post = new Float64Array(S * K);
61     for (let i = 0; i < S; i++) {
62       const z = (this.srts[i] - this.o.priorMean) / this.o.priorSd;
63       for (let k = 0; k < K; k++) this.post[i * K + k] = Math.exp(-0.5 * z * z) / K;
64     }
65     normalize(this.post);
66     this.lik = this.candidates.map((snr) => {
67       const l = new Float64Array(S * K);
68       for (let i = 0; i < S; i++) for (let k = 0; k < K; k++) l[i * K + k] = pCorrect(snr, this.srts[i], this.slopes[k],
69         this.o.lapse);
70       return l;
71     });
72   }
73
74   private srtMarginal(p: Float64Array): Float64Array {
75     const K = this.slopes.length;
76     const m = new Float64Array(this.srts.length);
77     for (let i = 0; i < m.length; i++) for (let k = 0; k < K; k++) m[i] += p[i * K + k];
78     return m;
```

src/engine/bayes.ts (continued)

```
78   }
79
80   nextSnr(): number {
81     let best = 0;
82     let bestH = Infinity;
83     const tmp = new Float64Array(this.post.length);
84     for (let c = 0; c < this.candidates.length; c++) {
85       const l = this.lik[c];
86       let pc = 0;
87       for (let j = 0; j < l.length; j++) pc += this.post[j] * l[j];
88       let h = 0;
89       for (const outcome of [true, false]) {
90         const po = outcome ? pc : 1 - pc;
91         if (po < 1e-9) continue;
92         for (let j = 0; j < l.length; j++) tmp[j] = (this.post[j] * (outcome ? l[j] : 1 - l[j])) / po;
93         h += po * entropy(this.srtMarginal(tmp));
94       }
95       if (h < bestH - 1e-12) {
96         bestH = h;
97         best = c;
98       }
99     }
100    return this.candidates[best];
101  }
102
103  update(trial: Trial): void {
104    const K = this.slopes.length;
105    for (let i = 0; i < this.srts.length; i++) {
106      for (let k = 0; k < K; k++) {
107        const p = pCorrect(trial.snr, this.srts[i], this.slopes[k], this.o.lapse);
108        this.post[i * K + k] *= trial.correct ? p : 1 - p;
109      }
110    }
111    normalize(this.post);
112    this.n++;
113  }
114
115  done(): boolean {
116    if (this.n >= this.o.maxTrials) return true;
117    return this.n >= this.o.minTrials && this.estimate().sd <= this.o.targetSd;
118  }
119
120  /** SRT marginal posterior (for plotting). */
121  marginal(): { srt: number; p: number }[] {
122    const m = this.srtMarginal(this.post);
123    return this.srts.map((srt, i) => ({ srt, p: m[i] }));
124  }
125
126  estimate(): Estimate {
127    const m = this.srtMarginal(this.post);
128    let mean = 0;
129    for (let i = 0; i < m.length; i++) mean += m[i] * this.srts[i];
130    let v = 0;
131    for (let i = 0; i < m.length; i++) v += m[i] * (this.srts[i] - mean) ** 2;
132    return { srt: mean, sd: Math.sqrt(v), lo90: quantile(m, this.srts, 0.05), hi90: quantile(m, this.srts, 0.95), trials:
this.n };
133  }
134 }
135
136 function normalize(p: Float64Array): void {
137   let s = 0;
138   for (let i = 0; i < p.length; i++) s += p[i];
139   for (let i = 0; i < p.length; i++) p[i] /= s;
140 }
141
142 function entropy(p: Float64Array): number {
143   let h = 0;
144   for (let i = 0; i < p.length; i++) if (p[i] > 0) h -= p[i] * Math.log(p[i]);
145   return h;
146 }
147
148 function quantile(p: Float64Array, xs: number[], q: number): number {
149   let c = 0;
150   for (let i = 0; i < p.length; i++) {
151     c += p[i];
152     if (c >= q) return xs[i];
153   }
154   return xs[xs.length - 1];
155 }
```

src/engine/family.ts

SHA256 06afc8d2d9fc522892f0619f31c9b21e805a601e70fd0390907b059504fe54f3

```
1 import type { Estimate } from "./types";
2
3 export interface Member {
4   id: string;
5   name: string;
6   estimate: Estimate;
7   takenAt: string;
8   device: string;
9 }
10
11 export type Verdict = "similar" | "harder" | "easier" | "too-few";
12
13 export interface Comparison {
14   member: Member;
15   reference: number;
16   delta: number;
17   verdict: Verdict;
18 }
19
20 /** Minimum gap (dB) before Hearsay calls a difference notable, regardless of measured precision. */
21 export const MIN_NOTABLE_DB = 3;
22
23 export function median(xs: number[]): number {
24   const s = [...xs].sort((a, b) => a - b);
25   const m = s.length >> 1;
26   return s.length % 2 ? s[m] : (s[m - 1] + s[m]) / 2;
27 }
28
29 /**
30  * Compare each member with the median of everyone else tested on the same device and headphones.
31  * Because everyone shares the same (uncalibrated) output chain, device-level offsets cancel out.
32  * A difference is notable only if it exceeds both MIN_NOTABLE_DB and 1.645x the combined uncertainty.
33  */
34 export function compareGroup(members: Member[]): Comparison[] {
35   return members.map((member) => {
36     const others = members.filter((m) => m.id !== member.id);
37     if (others.length === 0) return { member, reference: NaN, delta: NaN, verdict: "too-few" as const };
38     const reference = median(others.map((m) => m.estimate.srt));
39     const refSd = median(others.map((m) => m.estimate.sd));
40     const delta = member.estimate.srt - reference;
41     const threshold = Math.max(MIN_NOTABLE_DB, 1.645 * Math.hypot(member.estimate.sd, refSd));
42     const verdict: Verdict = delta > threshold ? "harder" : delta < -threshold ? "easier" : "similar";
43     return { member, reference, delta, verdict };
44   });
45 }
```

src/engine/headphones.ts

SHA256 acb2c55e02761d1a3eee83ed1dfdbe964f386ca87d9a2cd9bb3f178155ebb7d4

```
1 import type { Rng } from "./rng";
2
3 /**
4  * Headphone screening in the style of Woods et al. (2017): three 200 Hz tones, one of which is
5  * presented in antiphase between the ears. Over headphones the antiphase tone is not quieter;
6  * the target is the one that is actually 6 dB quieter. Over loudspeakers the antiphase tone
7  * partially cancels acoustically and tends to be picked instead.
8  */
9 export interface HeadphoneTrial {
10   quiet: number;
11   antiphase: number;
12 }
13
14 export function makeHeadphoneTrial(rng: Rng): HeadphoneTrial {
15   const quiet = Math.floor(rng() * 3);
16   let antiphase = Math.floor(rng() * 2);
17   if (antiphase >= quiet) antiphase++;
18   return { quiet, antiphase };
19 }
20
21 export const HEADPHONE_TRIALS = 6;
22 export const HEADPHONE_PASS = 5;
23
24 export function headphonePassed(correct: number): boolean {
25   return correct >= HEADPHONE_PASS;
26 }
```

src/engine/psychometric.ts

SHA256 6717fe1ce535efb67aec7bc59b9e7282da45aad6fa7dc3a72d6e3b77fa3d818e

```
1 /** Probability that a listener repeats a whole digit triplet correctly at a given SNR (dB). */
2 export interface Listener {
3   srt: number;
4   slope: number;
5   lapse: number;
6 }
7
8 export const DEFAULT_SLOPE = 0.7;
9 export const DEFAULT_LAPSE = 0.03;
10
11 export function pCorrect(snr: number, srt: number, slope = DEFAULT_SLOPE, lapse = DEFAULT_LAPSE): number {
12   return (1 - lapse) / (1 + Math.exp(-slope * (snr - srt)));
13 }
```


src/engine/rng.ts

SHA256 a8abba0d194d931ffed71e3f9e5f2bd0e4360a5928c5314a583743b613b2f750

```
1 export type Rng = () => number;
2
3 export function mulberry32(seed: number): Rng {
4   let a = seed >>> 0;
5   return () => {
6     a = (a + 0x6d2b79f5) >>> 0;
7     let t = a;
8     t = Math.imul(t ^ (t >>> 15), t | 1);
9     t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
10    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
11  };
12 }
13
14 export function gaussian(rng: Rng): number {
15   const u = Math.max(rng(), 1e-12);
16   const v = rng();
17   return Math.sqrt(-2 * Math.log(u)) * Math.cos(2 * Math.PI * v);
18 }
```

src/engine/simulate.ts

SHA256 90c49e10085766824a5200057c0ad0c754df7803ddd86e2e7dc76f591c957759

```
1 import { BayesianDin } from "./bayes";
2 import { pCorrect } from "./psychometric";
3 import { mulberry32 } from "./rng";
4 import { Staircase } from "./staircase";
5 import type { AdaptiveProcedure } from "./types";
6
7 export interface SimConfig {
8   listeners: number;
9   seed: number;
10  trueSlope: number;
11  trueLapse: number;
12  srtRange: [number, number];
13 }
14
15 export interface MethodStats {
16   method: string;
17   rmse: number;
18   bias: number;
19   meanTrials: number;
20   /** Fraction of runs whose reported 90% interval contained the true SRT. */
21   coverage90: number;
22   /** SD of the difference between two independent runs on the same simulated listener. */
23   retestSd: number;
24   errors: number[];
25   trials: number[];
26 }
27
28 export const DEFAULT_SIM: SimConfig = { listeners: 400, seed: 7, trueSlope: 0.7, trueLapse: 0.02, srtRange: [-14, 2] };
29
30 export type ProcedureFactory = () => AdaptiveProcedure;
31
32 export const METHODS: Record<string, ProcedureFactory> = {
33   staircase: () => new Staircase(),
34   staircase12: () => new Staircase({ trials: 12, discard: 2 }),
35   bayesian: () => new BayesianDin(),
36 };
37
38 export function runOnce(make: ProcedureFactory, srt: number, slope: number, lapse: number, rng: () => number) {
39   const proc = make();
40   while (!proc.done()) {
41     const snr = proc.nextSnr();
42     proc.update({ snr, correct: rng() < pCorrect(snr, srt, slope, lapse) });
43   }
44   return proc.estimate();
45 }
46
47 export function simulate(make: ProcedureFactory, cfg: SimConfig = DEFAULT_SIM): MethodStats {
48   const rng = mulberry32(cfg.seed);
49   const errors: number[] = [];
50   const trials: number[] = [];
51   const diffs: number[] = [];
52   let covered = 0;
53   let name = "";
54   for (let i = 0; i < cfg.listeners; i++) {
55     const srt = cfg.srtRange[0] + rng() * (cfg.srtRange[1] - cfg.srtRange[0]);
56     const a = runOnce(make, srt, cfg.trueSlope, cfg.trueLapse, rng);
57     const b = runOnce(make, srt, cfg.trueSlope, cfg.trueLapse, rng);
58     errors.push(a.srt - srt);
59     trials.push(a.trials);
60     diffs.push(a.srt - b.srt);
61     if (a.lo90 <= srt && srt <= a.hi90) covered++;
62     name ||= make().name;
63   }
64   const mean = (x: number[]) => x.reduce((s, v) => s + v, 0) / x.length;
65   const md = mean(diffs);
66   return {
67     method: name,
68     rmse: Math.sqrt(mean(errors.map((e) => e * e))),
69     bias: mean(errors),
70     meanTrials: mean(trials),
71     coverage90: covered / cfg.listeners,
72     retestSd: Math.sqrt(mean(diffs.map((d) => (d - md) ** 2))),
73     errors,
74     trials,
75   };
76 }
```

src/engine/staircase.ts

SHA256 97dd2ea440aed03e25eb962bd5494b1cbea568a1de8458fc8d9c48c12ee0f812

```
1 import type { AdaptiveProcedure, Estimate, Trial } from "./types";
2
3 export interface StaircaseOptions {
4   start?: number;
5   step?: number;
6   trials?: number;
7   discard?: number;
8   min?: number;
9   max?: number;
10 }
11
12 /**
13  * Classic fixed-length digits-in-noise track: 1-up/1-down with 2 dB steps.
14  * The SRT is the mean SNR of trials after `discard`, plus the level the next trial would have used.
15  */
16 export class Staircase implements AdaptiveProcedure {
17   readonly name: string;
18   private snr: number;
19   private readonly history: Trial[] = [];
20   private readonly o: Required<StaircaseOptions>;
21
22   constructor(opts: StaircaseOptions = {}) {
23     this.o = { start: 0, step: 2, trials: 24, discard: 4, min: -24, max: 12, ...opts };
24     this.snr = this.o.start;
25     this.name = `Staircase (${this.o.trials} trials)`;
26   }
27
28   nextSnr(): number {
29     return this.snr;
30   }
31
32   update(trial: Trial): void {
33     this.history.push(trial);
34     const next = trial.snr + (trial.correct ? -this.o.step : this.o.step);
35     this.snr = Math.max(this.o.min, Math.min(this.o.max, next));
36   }
37
38   done(): boolean {
39     return this.history.length >= this.o.trials;
40   }
41
42   estimate(): Estimate {
43     const levels = this.history.slice(this.o.discard).map((t) => t.snr);
44     levels.push(this.snr);
45     const mean = levels.reduce((a, b) => a + b, 0) / levels.length;
46     const variance = levels.reduce((a, b) => a + (b - mean) ** 2, 0) / Math.max(1, levels.length - 1);
47     const sd = Math.sqrt(variance / levels.length);
48     return { srt: mean, sd, lo90: mean - 1.645 * sd, hi90: mean + 1.645 * sd, trials: this.history.length };
49   }
50 }
```

src/engine/triplets.ts

SHA256 0661fbc043ddff3352d79acf40ee18fae75aa3504ead59309bfbf841844d3b4d

```
1 import type { Rng } from "../rng";
2
3 /** "7" is left out, as in several English digits-in-noise designs, because it is the only two-syllable digit. */
4 export const DIGITS = [0, 1, 2, 3, 4, 5, 6, 8, 9] as const;
5 export type Digit = (typeof DIGITS)[number];
6 export type Triplet = [Digit, Digit, Digit];
7
8 export function makeTriplet(rng: Rng): Triplet {
9   const pool: Digit[] = [...DIGITS];
10  const out: Digit[] = [];
11  for (let i = 0; i < 3; i++) {
12    const idx = Math.floor(rng() * pool.length);
13    out.push(pool.splice(idx, 1)[0]);
14  }
15  return out as Triplet;
16 }
17
18 export function parseResponse(text: string): number[] {
19   return [...text.replace(/\D/g, "")].map(Number);
20 }
21
22 export function scoreTriplet(target: Triplet, response: string): { correct: boolean; digitsCorrect: number } {
23   const r = parseResponse(response);
24   let digitsCorrect = 0;
25   for (let i = 0; i < 3; i++) if (r[i] === target[i]) digitsCorrect++;
26   return { correct: digitsCorrect === 3 && r.length === 3, digitsCorrect };
27 }
```

src/engine/types.ts

SHA256 597bb4b2ce7bcc97ece215ede631a7d03667390bec8b602ff7df16c7c506a405

```
1 export interface Trial {
2   snr: number;
3   correct: boolean;
4 }
5
6 export interface Estimate {
7   srt: number;
8   /** Uncertainty (posterior SD for Bayesian; SD of tracked SNRs / sqrt(n) for the staircase). */
9   sd: number;
10  lo90: number;
11  hi90: number;
12  trials: number;
13 }
14
15 export interface AdaptiveProcedure {
16   readonly name: string;
17   nextSnr(): number;
18   update(trial: Trial): void;
19   done(): boolean;
20   estimate(): Estimate;
21 }
```

SHA256 fdbf24b11bc4e6f2406bb0fba1b4b740634505dcacac3a8535b1e23cafd29f2

```

1 import "./style.css";
2 import { AudioEngine } from "./audio/player";
3 import { AUDIOGRAM_FREQS, PROFILES } from "./audio/hearloss";
4 import { BayesianDin } from "./engine/bayes";
5 import { compareGroup, type Member } from "./engine/family";
6 import { HEADPHONE_TRIALS, headphonePassed, makeHeadphoneTrial } from "./engine/headphones";
7 import { pCorrect } from "./engine/psychometric";
8 import { mulberry32 } from "./engine/rng";
9 import { DEFAULT_SIM, METHODS, simulate, type MethodStats } from "./engine/simulate";
10 import { makeTriplet, scoreTriplet, type Triplet } from "./engine/triplets";
11 import type { Estimate, Trial } from "./engine/types";
12 import { esc, familyChart, histogram, posteriorChart, profileChart, trackChart } from "./ui/charts";
13 import { demoFamily, loadMembers, saveMembers } from "./storage";
14 import { ICONS } from "./ui/icons";
15
16 const app = document.getElementById("app");
17 let audio: AudioEngine | null = null;
18 const engine = () => (audio ??= new AudioEngine());
19 const rng = mulberry32((Date.now() ^ 0x5eed) >>> 0);
20
21 const DISCLAIMER =
22   "Hearsay is an educational self-check, not a medical device. It cannot diagnose hearing loss or any condition. If you
   are worried about your hearing, see an audiologist or doctor.";
23
24 function shell(active: string, body: string): string {
25   const link = (href: string, label: string) => `<a href="#/${href}" class="${active === href ? "active" :
   ""}">${label}</a>`;
26   return `<header class="top"><a class="brand" href="#/${ICONS.ear}" Hearsay</a><nav>${link("check", "Take the
   check")}${link("family", "Family board")}${link("simulator", "Hear-through")}${link("lab", "Lab")}${link("about",
   "Honesty")}</nav></header>
27   <main>${body}</main>
28   <footer><p>${DISCLAIMER}</p><p>Speech synthesized with <a href="https://elevenlabs.io" target="_blank"
   rel="noopener">ElevenLabs</a>. No data leaves your browser.</p></footer>`;
29 }
30
31 function route(): void {
32   engine().stop();
33   const [path, query] = location.hash.replace(/^#\/?/, "").split("?");
34   const params = new URLSearchParams(query ?? "");
35   window.scrollTo(0, 0);
36   switch (path) {
37     case "check":
38       return checkView(params);
39     case "family":
40       return familyView();
41     case "simulator":
42       return simulatorView();
43     case "lab":
44       return labView();
45     case "about":
46       return aboutView();
47     default:
48       return homeView();
49   }
50 }
51
52 function homeView(): void {
53   app.innerHTML = shell(
54     "",
55     `<section class="hero">
56     <p class="eyebrow">AI for Human Health · UnivaBio 2026</p>
57     <h1>"What?" is the most expensive word at dinner.</h1>
58     <p class="lead">Hearsay helps families explore <b>hearing spoken digits in noise</b> through a short, adaptive
   headphone task. See how the noise level changes with your answers, then compare task results collected on the same setup.
   <b>This educational prototype has not been clinically validated.</b></p>
59     <div class="cta"><a class="btn primary" href="#/check">Start the 3-minute check</a><a class="btn"
   href="#/family">See the family board</a></div>
60     <p class="fine">${DISCLAIMER}</p>
61   </section>
62   <section class="grid3">
63     <article class="card"><h3>${ICONS.target}</h3> Adaptive, not a quiz</h3><p>A Bayesian engine picks every next noise
   level to learn the most about you. In simulation it reaches a <b>~34% lower error</b> than a classic staircase of the same
   length</b> and stops in about 12 triplets.</p></article>
64     <article class="card"><h3>${ICONS.headphones}</h3> Honest about your headphones</h3><p>The comparison uses the <b>same
   device, headphones and volume</b> to reduce a shared device-offset problem under the model. Fit, attention, language and
   the listening environment can still differ. This does not calibrate the headphones or establish clinical
   validity.</p></article>
65     <article class="card"><h3>${ICONS.ear}</h3> Hear through their ears</h3><p>An illustrative simulator lets the rest of

```

src/main.ts (continued)

```
the family hear why "just listen harder" doesn't work – and why speaking clearly beats speaking loudly.</p></article>
66   </section>
67   <section class="how">
68     <h2>How the check works</h2>
69     <ol class="steps"><li><b>Volume</b></li><li><b>Headphone check</b></li><li><b>Headphone check</b></li><li><b>Headphone check</b></li></ol>
    brief tone task checks the headphone setup; it cannot verify equipment or calibrate listening
    levels.</span></li><li><b>Digits in noise</b></li><li><b>Digits in noise</b></li><li><b>Digits in noise</b></li><li><b>Digits in noise</b></li></ol>
    you.</span></li><li><b>Compare</b></li><li><b>Compare</b></li><li><b>Compare</b></li><li><b>Compare</b></li><li><b>Compare</b></li></ol>
    shown.</span></li><li><b>Compare</b></li><li><b>Compare</b></li><li><b>Compare</b></li><li><b>Compare</b></li><li><b>Compare</b></li></ol>
70   </section>`,
71   );
72 }
73
74 interface CheckState {
75   name: string;
76   device: string;
77   virtualSrt: number | null;
78 }
79
80 function checkView(params: URLSearchParams): void {
81   const virtual = params.get("virtual");
82   const st: CheckState = { name: "", device: "", virtualSrt: virtual !== null && virtual !== "" ? Number(virtual) : null
    };
83   const lastDevice = loadMembers().at(-1)?.device ?? "";
84   app.innerHTML = shell(
85     "check",
86     `<section class="panel narrow">
87       <p class="eyebrow">Step 1 of 4 · Who's listening?</p>
88       <h2>Let's get you set up</h2>
89       ${st.virtualSrt !== null ? `<p class="notice">Demo mode: a <b>virtual listener</b> (true SRT ${st.virtualSrt} dB)
    will answer automatically. Results are simulated.</p>` : ""}
90       <label>Your name (stays on this device)<input id="name" autocomplete="off" placeholder="e.g. Grandpa"
    value="${st.virtualSrt !== null ? "Virtual listener" : ""}</input></label>
91       <label>Device + headphones<input id="device" placeholder="e.g. Mom's laptop + white earbuds"
    value="${esc(lastDevice || (st.virtualSrt !== null ? "Simulated" : ""))}</input></label>
92       <p class="fine">Family comparisons only make sense when everyone uses the same device, headphones and volume.</p>
93       <button class="btn primary" id="go">Continue</button>
94     </section>`,
95   );
96   document.getElementById("go")!.onclick = () => {
97     st.name = (document.getElementById("name") as HTMLInputElement).value.trim() || "Listener";
98     st.device = (document.getElementById("device") as HTMLInputElement).value.trim() || "Unnamed device";
99     engine().load().catch(() => undefined);
100     if (st.virtualSrt !== null) return runTest(st);
101     volumeStep(st);
102   };
103 }
104
105 function volumeStep(st: CheckState): void {
106   const main = app.querySelector("main")!;
107   main.innerHTML = `<section class="panel narrow">
108     <p class="eyebrow">Step 2 of 4 · Volume</p>
109     <h2>Put on headphones and set a comfortable level</h2>
110     <p>Press play. You'll hear three digits clearly over soft noise. Adjust until it's comfortable – not loud. <b>Don't
    change it again</b> during the check.</p>
111     <div class="row"><button class="btn" id="play">▶ Play sample</button><input type="range" id="vol" min="0.05" max="1"
    step="0.01" value="0.5" aria-label="Volume"/></div>
112     <button class="btn primary" id="next">It's comfortable →</button>
113   </section>`;
114   const vol = document.getElementById("vol") as HTMLInputElement;
115   vol.oninput = () => engine().setVolume(Number(vol.value));
116   document.getElementById("play")!.onclick = async () => {
117     await engine().load();
118     await engine().playTriplet(makeTriplet(rng), 6);
119   };
120   document.getElementById("next")!.onclick = () => headphoneStep(st);
121 }
122
123 function headphoneStep(st: CheckState): void {
124   let i = 0;
125   let correct = 0;
126   const main = app.querySelector("main")!;
127   const trial = () => {
128     const t = makeHeadphoneTrial(rng);
129     main.innerHTML = `<section class="panel narrow">
130       <p class="eyebrow">Step 3 of 4 · Headphone check (${i + 1}/${HEADPHONE_TRIALS})</p>
131       <h2>Which tone was the quietest?</h2>
132       <p>Three low hums will play. Choose the quietest. This tone task cannot verify equipment type or calibrate
    listening levels.</p>
133       <button class="btn" id="play">▶ Play the three tones</button>`
```

src/main.ts (continued)

```
134     <div class="choices">${[1, 2, 3].map((n) => `<button class="btn choice" data-c="${n - 1}"`  
disabled>${n}</button>`).join("")}</div>  
135     <button class="link" id="skip">Skip the tone task</button>  
136   </section>`;   
137   const choices = [...main.querySelectorAll<HTMLButtonElement>(".choice")];  
138   document.getElementById("play")!.onclick = async () => {  
139     await engine().playHeadphoneTrial(t.quiet, t.antiphase);  
140     choices.forEach((c) => (c.disabled = false));  
141   };  
142   choices.forEach(  
143     (c) =>  
144       (c.onclick = () => {  
145         if (Number(c.dataset.c) === t.quiet) correct++;  
146         i++;  
147         if (i < HEADPHONE_TRIALS) return trial();  
148         if (headphonePassed(correct)) return runTest(st, true);  
149         main.innerHTML = `<section class="panel narrow"><h2>Tone-task criterion not met</h2><p>You got  
${correct}/${HEADPHONE_TRIALS}. This response pattern does not identify your equipment. You can retry the tone task or  
continue with that limitation noted.</p>  
150         <div class="row"><button class="btn primary" id="retry">Retry</button><button class="btn" id="anyway">Continue  
without meeting the tone-task criterion</button></div></section>`;   
151         document.getElementById("retry")!.onclick = () => headphoneStep(st);  
152         document.getElementById("anyway")!.onclick = () => runTest(st, false);  
153       }  
154     );  
155     document.getElementById("skip")!.onclick = () => runTest(st, false);  
156   };  
157   trial();  
158 }  
159  
160 function runTest(st: CheckState, verified = false): void {  
161   const proc = new BayesianDin();  
162   const trials: Trial[] = [];  
163   const main = app.querySelector("main")!;  
164   const vrng = mulberry32(42);  
165   let target: Triplet = makeTriplet(rng);  
166   let snr = proc.nextSnr();  
167   let typed = "";  
168  
169   const render = () => {  
170     const est = proc.estimate();  
171     main.innerHTML = `<section class="panel test">  
172       ${st.virtualSrt !== null ? `<p class="notice">Synthetic demo: responses and this result are simulated. No person's  
hearing is assessed.</p>` : ""}  
173       <div class="testhead"><p class="eyebrow">Step 4 of 4 · Digits in noise · triplet ${trials.length + 1}</p><div  
class="meter"><span style="width:${Math.min(100, (trials.length / 14) * 100)}%"></span></div></div>  
174       <div class="testgrid">  
175         <div>  
176           <h2>Type the three digits you heard</h2>  
177           <p class="fine">Guess if unsure. Every answer teaches the engine something.</p>  
178           <div class="display" aria-live="polite">${[0, 1, 2].map((k) => `<span class="${typed[k] ? "filled" :  
""}">${typed[k] ?? "."}</span>`).join("")}</div>  
179           <div class="keypad">${[1, 2, 3, 4, 5, 6, 8, 9, 0].map((d) => `<button class="key"  
data-d="${d}">${d}</button>`).join("")}<button class="key alt" data-d="back">⌫</button><button class="key alt"  
id="replay">↺</button><button class="key submit" id="submit" ${typed.length === 3 ? "" : "disabled"}>OK</button></div>  
180         </div>  
181         <aside class="belief">  
182           <h3>What Hearsay believes right now</h3>  
183           ${posteriorChart(proc.marginal(), est)}  
184           <p class="fine">Shaded band = the model's 90% credible interval for the task threshold. Current estimate:  
185           <b>${est.srt.toFixed(1)} dB SNR</b>; posterior SD: ${est.sd.toFixed(1)} dB. After at least 10 triplets, the run stops at  
SD ≤ 1.25 dB, or at 24 triplets.</p>  
186           ${trials.length ? trackChart(trials) : ""}  
187         </aside>  
188       </section>`;   
189     main.querySelectorAll<HTMLButtonElement>(".key[data-d]").forEach(  
190       (k) =>  
191         (k.onclick = () => {  
192           if (k.dataset.d === "back") typed = typed.slice(0, -1);  
193           else if (typed.length < 3) typed += k.dataset.d;  
194           render();  
195         }  
196       );  
197     document.getElementById("replay")!.onclick = () => play();  
198     document.getElementById("submit")!.onclick = () => submit();  
199   };  
200  
201   const play = async () => {  
202     await engine().load();
```


src/main.ts (continued)

```
203     await engine().playTriplet(target, snr);
204   };
205
206   const submit = () => {
207     const { correct } = scoreTriplet(target, typed);
208     const trial = { snr, correct };
209     trials.push(trial);
210     proc.update(trial);
211     typed = "";
212     if (proc.done()) return finish(st, proc.estimate(), trials, verified || st.virtualSrt !== null);
213     target = makeTriplet(rng);
214     snr = proc.nextSnr();
215     render();
216     void next();
217   };
218
219   const onKey = (e: KeyboardEvent) => {
220     if (!document.querySelector(".keypad")) return window.removeEventListener("keydown", onKey);
221     if (/^[0-9]$/.test(e.key) && e.key !== "7" && typed.length < 3) typed += e.key;
222     else if (e.key === "Backspace") typed = typed.slice(0, -1);
223     else if (e.key === "Enter" && typed.length === 3) return submit();
224     else return;
225     render();
226   };
227   window.addEventListener("keydown", onKey);
228
229   const next = async () => {
230     const played = play().catch(() => undefined);
231     if (st.virtualSrt === null) return;
232     await Promise.race([played, new Promise((r) => setTimeout(r, 1400))]);
233     const ok = vrng() < pCorrect(snr, st.virtualSrt, 0.7, 0.02);
234     const answer = ok ? target.join("") : target.map((d, k) => (k === 1 ? (d === 9 ? 8 : 9) : d)).join("");
235     for (const ch of answer) {
236       await new Promise((r) => setTimeout(r, 160));
237       typed += ch;
238       render();
239     }
240     await new Promise((r) => setTimeout(r, 250));
241     if (document.getElementById("submit")) submit();
242   };
243
244   render();
245   void next();
246 }
247
248 function finish(st: CheckState, est: Estimate, trials: Trial[], verified: boolean): void {
249   const members = loadMembers();
250   const member: Member = { id: `m-${Date.now()}`, name: st.name, estimate: est, takenAt: new Date().toISOString(),
251     device: st.device + (verified ? "" : " (headphones not verified)") };
252   const main = app.querySelector("main")!;
253   const sameDevice = members.filter((m) => m.device.replace(" (headphones not verified)", "") === st.device);
254   main.innerHTML = `<section class="panel">
255     <p class="eyebrow">Done in ${trials.length} triplets</p>
256     ${st.virtualSrt !== null ? `<p class="notice">Synthetic demo: responses and this result are simulated. No person's
257     hearing is assessed.</p>` : ""}
258     <h2>${esc(st.name)}, your model-estimated task threshold is <span class="big">${est.srt.toFixed(1)} dB
259     SNR</span></h2>
260     <p class="lead">This estimates the signal-to-noise ratio around which the model predicts half the digit triplets will
261     be correct. Lower values indicate better performance on this task, not a clinical hearing category. Model-based 90%
262     credible interval: ${est.lo90.toFixed(1)} to ${est.hi90.toFixed(1)} dB.</p>
263     <div class="grid2"><div>${trackChart(trials)}<p class="fine">Green = got all three digits, red = missed. The engine
264     moved the noise to where your answers were most informative.</p></div>
265     <div class="card"><h3>What does this mean?</h3><p>Headphones, volume and listening conditions can affect this
266     estimate. There is no clinical 'normal range' here.</p><p>You can explore differences between results collected on
267     <b>this same setup</b> (${sameDevice.length} saved so far). Changes in fit, attention, language or environment can affect
268     scores; these comparisons have not been clinically validated.</p>
269     ${verified ? "" : `<p class="notice">The tone task was skipped or its criterion was not met. This task never verifies
270     equipment or calibrates listening levels.</p>`}</div></div>
271     <div class="row"><button class="btn primary" id="save">Save to family board</button><a class="btn"
272     href="#/check">Next person</a><button class="link" id="discard">Discard</button></div>
273     <p class="fine">${DISCLAIMER}</p>
274   </section>`;
275   document.getElementById("save")!.onclick = () => {
276     saveMembers([...members, member]);
277     location.hash = "#/family";
278   };
279   document.getElementById("discard")!.onclick = () => (location.hash = "#/");
280 }
281
282 function familyView(): void {
```

src/main.ts (continued)

```
272 const members = loadMembers();
273 const devices = [...new Set(members.map((m) => m.device.replace(" (headphones not verified)", "")))]);
274 const groups = devices.map((d) => ({ device: d, rows: compareGroup(members.filter((m) => m.device.replace(" (headphones
not verified)", "") === d)) }));
275 const verdictText: Record<string, string> = {
276   harder: "Found it noticeably harder than others on this device. Worth mentioning to a doctor or getting a
professional hearing test.",
277   easier: "Did noticeably better than others on this device.",
278   similar: "Similar to the others on this device.",
279   "too-few": "Add at least one more person on this device to compare.",
280 };
281 app.innerHTML = shell(
282   "family",
283   `<section class="panel">
284     <p class="eyebrow">Same device, same headphones, same volume</p>
285     <h2>Family board</h2>
286     <p class="lead">Results collected on the same setup are compared with the <b>median of everyone else</b>. Under the
model, this reduces a shared device-offset problem; fit, attention, language and environment can still affect the
comparison. The prototype flags a gap above <b>3 dB</b> and its estimated uncertainty. This rule is an educational design
choice, not a clinically validated threshold.</p>
287     ${
288       members.length
289       ? groups
290         .map(
291           (g) => `<div class="group"><h3>${ICONS.headphones} ${esc(g.device)}</h3>${familyChart(g.rows)}
292           <ul class="verdicts">${g.rows
293             .map(
294               (r) => `<li class="${r.verdict}"><b>${esc(r.member.name)}</b> – ${r.member.estimate.srt.toFixed(1)}
dB${Number.isFinite(r.delta) ? ` (${r.delta} >= 0 ? "+" : "")${r.delta.toFixed(1)} vs. others` : ""}.
295               ${verdictText[r.verdict]}<button class="link del" data-id="${r.member.id}">remove</button></li>`,
296               .join("")</ul></div>`,
297             )
298             .join("")
299             : `<div class="empty"><p>No one yet. Take the check, pass the headphones to the next person, repeat.</p></div>`
300         }
301     <div class="row"><a class="btn primary" href="#/check">Add a person</a><button class="btn" id="demo">Load synthetic
demo family</button>${members.length ? `<button class="link" id="clear">Clear board</button>` : ""}</div>
302     <p class="fine">The demo family is invented for illustration and labelled “(synthetic)”. ${DISCLAIMER}</p>
303   </section>`,
304 );
305 document.getElementById("demo")!.onclick = () => {
306   saveMembers([...loadMembers().filter((m) => !m.id.startsWith("demo-")), ...demoFamily()]);
307   familyView();
308 };
309 document.getElementById("clear")?.addEventListener("click", () => {
310   saveMembers([]);
311   familyView();
312 });
313 app.querySelectorAll<HTMLButtonElement>(".del").forEach(
314   (b) =>
315     (b.onclick = () => {
316       saveMembers(loadMembers().filter((m) => m.id !== b.dataset.id));
317       familyView();
318     }),
319 );
320 }
321
322 function simulatorView(): void {
323   let current = PROFILES[1];
324   const render = () => {
325     app.innerHTML = shell(
326       "simulator",
327       `<section class="panel">
328         <p class="eyebrow">Empathy mode · illustrative only</p>
329         <h2>Hear through their ears</h2>
330         <p class="lead">Most hearing loss doesn't make the world quieter – it makes it <b>blurrier</b>. High pitches fade
first, taking consonants with them. Play the same message both ways.</p>
331         <div class="profiles">${PROFILES.slice(1)
332           .map((p) => `<button class="chip ${p.id} === current.id ? "on" : ""` data-p="${p.id}">${p.label}</button>`)
333           .join("")}</div>
334         <div class="grid2"><div>${profileChart(current.loss, AUDIOGRAM_FREQS)}<p class="fine">${current.description}
These curves are hand-made teaching examples, not anyone's real audiogram. Filtering is an approximation: real hearing
loss also affects loudness growth and timing, which this does not model.</p></div>
335         <div class="card"><h3>“Hi Grandpa! I'm picking you up at 4:15 on Thursday...”</h3>
336         <div class="row"><button class="btn" id="orig">▶ Original</button><button class="btn primary" id="sim">▶
Through “${current.label}”</button></div>
337         <div class="row"><button class="btn" id="trip">▶ Digits in noise, through “${current.label}”</button></div>
338         <p class="fine">Tip for families: face the person, reduce background noise, and slow down. Shouting mostly
makes vowels louder – the part they already hear.</p></div></div>
```

src/main.ts (continued)

```
339     </section>`,
340   );
341   app.querySelectorAll<HTMLButtonElement>(".chip").forEach(
342     (c) => {
343       (c.onclick = () => {
344         current = PROFILES.find((p) => p.id === c.dataset.p)!;
345         render();
346       }),
347     );
348   document.getElementById("orig")!.onclick = async () => {
349     await engine().load();
350     await engine().playSentence(null);
351   };
352   document.getElementById("sim")!.onclick = async () => {
353     await engine().load();
354     await engine().playSentence(current);
355   };
356   document.getElementById("trip")!.onclick = async () => {
357     await engine().load();
358     await engine().playTripletThrough(current, makeTriplet(rng), 0);
359   };
360 };
361 render();
362 }
363
364 function labView(): void {
365   app.innerHTML = shell(
366     "lab",
367     `<section class="panel">
368       <p class="eyebrow">Measurable, reproducible, in your browser</p>
369       <h2>The Lab: does the AI actually help?</h2>
370       <p class="lead">We simulate hundreds of virtual listeners with known thresholds, run each test procedure on them
371       twice, and measure the error. Seeded, so you get the same numbers every time.</p>
372       <div class="row"><label class="inline">Virtual listeners <select id="n"><option>100</option><option
373       selected>400</option><option>1000</option></select></label>
374       <label class="inline">True slope (dB) <select id="slope"><option>0.5</option><option
375       selected>0.7</option><option>1.0</option></select></label>
376       <button class="btn primary" id="run">Run simulation</button><a class="btn" href="#/check?virtual=-4">Watch a
377       virtual listener take the check</a></div>
378       <div id="out"><p class="fine">Press run (takes a couple of seconds).</p></div>
379     </section>`,
380   );
381   const run = () => {
382     const out = document.getElementById("out")!;
383     out.innerHTML = `<p class="fine">Simulating...</p>`;
384     setTimeout(() => {
385       const listeners = Number((document.getElementById("n") as HTMLSelectElement).value);
386       const trueSlope = Number((document.getElementById("slope") as HTMLSelectElement).value);
387       const res: MethodStats[] = Object.values(METHODS).map((m) => simulate(m, { ...DEFAULT_SIM, listeners, trueSlope
388     }));
389       const [s24, s12, bayes] = res;
390       out.innerHTML = `<table class="results"><thead><tr><th>Procedure</th><th>Avg. triplets</th><th>RMSE
391       (dB)</th><th>Bias (dB)</th><th>Test-retest SD (dB)</th><th>90% interval really contains truth</th></tr></thead><tbody>
392       <tr><td>${res.map((r) => `<tr class="${r === bayes ? "hl" :
393       ""}"><td>${r.method}</td><td>${r.meanTrials.toFixed(1)}</td><td>${r.rmse.toFixed(2)}</td><td>${r.bias >= 0 ? "+" :
394       ""}</td><td>${r.bias.toFixed(2)}</td><td>${r.retestSd.toFixed(2)}</td><td>${(r.coverage90 *
395       100).toFixed(0)}%</td></tr>`).join("")}</tr>
396       </tbody></table>
397       <div class="grid3 stats"><div class="stat"><b>${((1 - bayes.rmse / s12.rmse) * 100).toFixed(0)}%</b><span>lower
398       error than a staircase of the same length</span></div>
399       <div class="stat"><b>${((1 - bayes.meanTrials / s24.meanTrials) * 100).toFixed(0)}%</b><span>fewer triplets than
400       the classic 24-trial track</span></div>
401       <div class="stat"><b>${(bayes.coverage90 * 100).toFixed(0)}%</b><span>of Hearsay's "90%" intervals contained the
402       truth (staircase: ${(s24.coverage90 * 100).toFixed(0)}%</span></div></div>
403       <div class="grid2"><div><h3>Error distribution – Hearsay</h3>${histogram(bayes.errors, -5, 5, 20, "bar
404       b")}</div><div><h3>Error distribution – 12-trial staircase</h3>${histogram(s12.errors, -5, 5, 20, "bar s")}</div></div>
405       <p class="fine">Trade-off, stated plainly: the full 24-trial staircase is still slightly more precise (RMSE
406       ${s24.rmse.toFixed(2)} vs ${bayes.rmse.toFixed(2)} dB) – it just takes twice as long and its error bars are over-
407       confident. Simulated listeners follow a logistic psychometric function with 2% lapses; real people also get tired,
408       distracted and learn, which this does not capture.</p>`;
409     }, 30);
410   };
411   document.getElementById("run")!.onclick = run;
412 }
413
414 function aboutView(): void {
415   app.innerHTML = shell(
416     "about",
417     `<section class="panel prose">
418       <p class="eyebrow">Honesty page</p>
419     `
420   );
421 }
```

src/main.ts (continued)

```
403     <h2>What's real, what's synthetic, what's missing</h2>
404     <h3>What Hearsay is</h3><p>A browser-based educational self-check of <b>speech understanding in noise</b>, using a
digits-in-noise design (three spoken digits in speech-shaped noise, antiphasic presentation), a Bayesian adaptive
procedure, and same-device family comparison.</p>
405     <h3>What it is not</h3><ul><li>Not a diagnosis, screening device, or substitute for an audiologist. It has <b>not
been clinically validated</b>.</li><li>Not calibrated: without a calibrated headphone and level, absolute scores are not
comparable across devices. That's why there's no "normal range" on the results page.</li><li>Not suitable for children,
people with tinnitus flare-ups from sound, or anyone for whom sound exposure is uncomfortable.</li></ul>
406     <h3>Data provenance</h3><ul><li><b>Speech:</b> nine digit words and one sentence synthesized with the ElevenLabs
text-to-speech API (premade voice "Alice"). Generated by <code>scripts/make-stimuli.ts</code>.</li><li><b>Noise:</b>
stationary noise computed locally from the digits' long-term spectrum (random phase, seeded). No third-party
audio.</li><li><b>Validation data:</b> 100% synthetic – virtual listeners simulated in
<code>src/engine/simulate.ts</code>. No human-subject data was collected.</li><li><b>Demo family:</b> invented names and
numbers, labelled "(synthetic)".</li><li><b>Your results:</b> stored only in this browser's localStorage. No analytics,
no server.</li></ul>
407     <h3>Known limitations</h3><ul><li>Psychometric slope and lapse rate are assumptions; the engine marginalizes over
three slopes to be robust, and the Lab shows how accuracy changes when the true slope differs.</li><li>Synthetic speech
is not the same as recorded clinical stimuli; digits were RMS-equalized but not individually intelligibility-balanced in
noise.</li><li>The hear-through simulator is a filter-based teaching approximation.</li><li>Family comparison assumes
everyone keeps the same device, headphones and volume.</li></ul>
408     <h3>Built with</h3><p>TypeScript, Vite, Web Audio API, Vitest. Solo student project for the UnivaBio 2026
hackathon.</p>
409     <h3>AI use</h3><p>Code, tests and docs were generated with the Devin AI coding agent (Cognition) for Sharon
Basovich's UnivaBio entry. The only external AI service is ElevenLabs text-to-speech, called once offline to make the
digit recordings; the live site makes no AI or network calls beyond loading its own files. The adaptive engine is
ordinary Bayesian statistics running in your browser, not a machine-learning model.</p>
410   </section>,
411   );
412 }
413
414 window.addEventListener("hashchange", route);
415 route();
```

src/storage.ts

SHA256 7c700df4533b563e01cfbaf6e6b809e862e24a0606b136ff2f2dfaf15a1407e7

```
1 import type { Member } from "../engine/family";
2
3 const KEY = "hearsay.members.v1";
4
5 export function loadMembers(): Member[] {
6   try {
7     const raw = localStorage.getItem(KEY);
8     return raw ? (JSON.parse(raw) as Member[]) : [];
9   } catch {
10    return [];
11  }
12 }
13
14 export function saveMembers(members: Member[]): void {
15   localStorage.setItem(KEY, JSON.stringify(members));
16 }
17
18 /** Clearly labelled synthetic demo family, generated from the simulator – not real people. */
19 export function demoFamily(): Member[] {
20   const now = new Date().toISOString();
21   const mk = (id: string, name: string, srt: number, sd: number, trials: number): Member => ({
22     id,
23     name,
24     estimate: { srt, sd, lo90: srt - 1.645 * sd, hi90: srt + 1.645 * sd, trials },
25     takenAt: now,
26     device: "Demo laptop + wired earbuds (synthetic)",
27   });
28   return [
29     mk("demo-sharon", "Sharon (synthetic)", -11.2, 1.1, 12),
30     mk("demo-mom", "Mom (synthetic)", -10.4, 1.2, 11),
31     mk("demo-dad", "Dad (synthetic)", -9.6, 1.2, 12),
32     mk("demo-grandpa", "Grandpa (synthetic)", -3.1, 1.3, 13),
33   ];
34 }
```

SHA256 a167426cf8928057f504f9adff3c1c7f6e42b310ae855824939567ce0fe5313b

```
1 :root {
2   --bg: #fbf7f1;
3   --ink: #1f2430;
4   --muted: #5f6675;
5   --card: #ffffff;
6   --line: #e7e0d5;
7   --accent: #e0573b;
8   --accent2: #2f6f73;
9   --ok: #2f9e6d;
10  --miss: #d8443a;
11  --soft: #fdeee8;
12  font-family: "Inter", ui-sans-serif, system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
13  color: var(--ink);
14  background: var(--bg);
15 }
16 * { box-sizing: border-box; }
17 body { margin: 0; }
18 a { color: var(--accent2); }
19 .top { display: flex; align-items: center; justify-content: space-between; padding: 14px 32px; border-bottom: 1px solid
var(--line); background: rgba(251, 247, 241, 0.9); position: sticky; top: 0; backdrop-filter: blur(6px); z-index: 5;
flex-wrap: wrap; gap: 8px; }
20 .brand { font-weight: 800; font-size: 1.25rem; text-decoration: none; color: var(--ink); }
21 nav { display: flex; gap: 4px; flex-wrap: wrap; }
22 nav a { text-decoration: none; color: var(--muted); padding: 8px 12px; border-radius: 999px; font-weight: 600; font-size:
0.95rem; }
23 nav a.active, nav a:hover { background: var(--soft); color: var(--accent); }
24 main { max-width: 1080px; margin: 0 auto; padding: 32px 24px 16px; }
25 footer { max-width: 1080px; margin: 0 auto; padding: 16px 24px 40px; color: var(--muted); font-size: 0.85rem; border-top:
1px solid var(--line); }
26 h1 { font-size: clamp(2rem, 4.5vw, 3.4rem); line-height: 1.05; margin: 8px 0 16px; letter-spacing: -0.02em; }
27 h2 { font-size: 1.8rem; margin: 4px 0 12px; letter-spacing: -0.01em; }
28 h3 { margin: 0 0 8px; }
29 .eyebrow { text-transform: uppercase; letter-spacing: 0.12em; font-size: 0.75rem; font-weight: 700; color: var(--accent);
margin: 0; }
30 .lead { font-size: 1.12rem; line-height: 1.6; color: #333a48; max-width: 780px; }
31 .fine { font-size: 0.85rem; color: var(--muted); line-height: 1.5; }
32 .hero { padding: 24px 0 8px; }
33 .cta, .row { display: flex; gap: 12px; flex-wrap: wrap; align-items: center; margin: 16px 0; }
34 .btn, .key, .chip, .link { font-family: inherit; }
35 .btn { display: inline-flex; align-items: center; gap: 6px; padding: 12px 20px; border-radius: 12px; border: 1.5px solid
var(--line); background: var(--card); color: var(--ink); font-weight: 700; font-size: 1rem; cursor: pointer; text-
decoration: none; transition: transform 0.08s, box-shadow 0.15s; }
36 .btn:hover { transform: translateY(-1px); box-shadow: 0 4px 14px rgba(0, 0, 0, 0.07); }
37 .btn.primary { background: var(--accent); border-color: var(--accent); color: white; }
38 .btn.disabled { opacity: 0.4; cursor: default; transform: none; }
39 .link { background: none; border: 0; color: var(--muted); text-decoration: underline; cursor: pointer; font-size: 0.9rem;
}
40 .grid3 { display: grid; grid-template-columns: repeat(auto-fit, minmax(240px, 1fr)); gap: 16px; margin: 24px 0; }
41 .grid2 { display: grid; grid-template-columns: repeat(auto-fit, minmax(320px, 1fr)); gap: 20px; margin: 16px 0; align-
items: start; }
42 .card, .panel { background: var(--card); border: 1px solid var(--line); border-radius: 18px; padding: 22px; }
43 .panel { padding: 32px; box-shadow: 0 10px 40px rgba(60, 40, 20, 0.05); }
44 .panel.narrow { max-width: 620px; margin: 0 auto; }
45 .card p { line-height: 1.55; color: #3a4150; }
46 .how { margin: 32px 0; }
47 .steps { list-style: none; padding: 0; display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap:
12px; counter-reset: s; }
48 .steps li { counter-increment: s; background: var(--card); border: 1px solid var(--line); border-radius: 14px; padding:
16px 16px 16px 56px; position: relative; display: flex; flex-direction: column; gap: 4px; }
49 .steps li::before { content: counter(s); position: absolute; left: 16px; top: 16px; width: 28px; height: 28px; border-
radius: 50%; background: var(--accent2); color: white; display: grid; place-items: center; font-weight: 800; }
50 .steps span { color: var(--muted); font-size: 0.92rem; }
51 label { display: flex; flex-direction: column; gap: 6px; font-weight: 600; margin: 14px 0; }
52 label.inline { flex-direction: row; align-items: center; margin: 0; }
53 input, select { font: inherit; padding: 12px 14px; border-radius: 10px; border: 1.5px solid var(--line); background:
#fffffa; }
54 input[type="range"] { flex: 1; min-width: 200px; accent-color: var(--accent); padding: 0; }
55 .notice { background: #fff6d8; border: 1px solid #f0dc97; padding: 10px 14px; border-radius: 10px; font-size: 0.92rem; }
56 .choices { display: flex; gap: 12px; margin: 20px 0; }
57 .choice { width: 72px; height: 72px; justify-content: center; font-size: 1.6rem; }
58 .testhead { display: flex; justify-content: space-between; align-items: center; gap: 16px; }
59 .meter { flex: 0 0 200px; height: 8px; background: var(--line); border-radius: 99px; overflow: hidden; }
60 .meter span { display: block; height: 100%; background: var(--accent2); transition: width 0.3s; }
61 .testgrid { display: grid; grid-template-columns: minmax(280px, 380px) 1fr; gap: 32px; margin-top: 16px; }
62 @media (max-width: 800px) { .testgrid { grid-template-columns: 1fr; } }
63 .display { display: flex; gap: 10px; margin: 16px 0; }
64 .display span { width: 72px; height: 84px; border-radius: 14px; border: 2px dashed var(--line); display: grid; place-
items: center; font-size: 2.4rem; font-weight: 800; color: #c8afb2; }
```

src/style.css (continued)

```
65 .display span.filled { border-style: solid; border-color: var(--accent2); color: var(--ink); background: #f1f8f7; }
66 .keypad { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px; max-width: 300px; }
67 .key { height: 60px; border-radius: 14px; border: 1.5px solid var(--line); background: #fff; font-size: 1.4rem; font-
weight: 700; cursor: pointer; }
68 .key:hover { background: var(--soft); }
69 .key.alt { color: var(--muted); }
70 .key.submit { background: var(--accent); color: #fff; border-color: var(--accent); }
71 .key.submit:disabled { opacity: 0.35; }
72 .belief { background: #f7faf9; border: 1px solid #dbe9e7; border-radius: 16px; padding: 18px; }
73 .chart { width: 100%; height: auto; display: block; margin: 8px 0; }
74 .chart .area { fill: rgba(47, 111, 115, 0.18); }
75 .chart .line { fill: none; stroke: var(--accent2); stroke-width: 2.5; }
76 .chart .band { fill: rgba(224, 87, 59, 0.12); }
77 .chart .mean { stroke: var(--accent); stroke-width: 2; stroke-dasharray: 4 3; }
78 .chart .axis { stroke: #b8b0a3; }
79 .chart .grid { stroke: #eee8de; }
80 .chart .tick { font-size: 11px; fill: var(--muted); text-anchor: middle; }
81 .chart .label { font-size: 13px; font-weight: 700; fill: var(--ink); }
82 .chart .track { fill: none; stroke: #c9c1b4; stroke-width: 1.5; }
83 .chart circle.ok { fill: var(--ok); }
84 .chart circle.miss { fill: var(--miss); }
85 .chart .ci { stroke-width: 6; stroke-linecap: round; stroke: #cfd8d7; }
86 .chart .ci.harder { stroke: #f4c2b6; }
87 .chart .dot { fill: var(--accent2); stroke: white; stroke-width: 2; }
88 .chart .dot.harder { fill: var(--accent); }
89 .chart .dot.easier { fill: var(--ok); }
90 .chart .bar.b { fill: var(--accent2); }
91 .chart .bar.s { fill: #b9ada0; }
92 .big { color: var(--accent); white-space: nowrap; }
93 .group { margin: 20px 0; padding: 18px; border: 1px solid var(--line); border-radius: 14px; background: #fffdfa; }
94 .verdicts { list-style: none; padding: 0; margin: 8px 0 0 8px; display: grid; gap: 8px; }
95 .verdicts li { padding: 10px 14px; border-radius: 10px; background: #f4f2ee; line-height: 1.45; }
96 .verdicts li.harder { background: var(--soft); border-left: 4px solid var(--accent); }
97 .verdicts li.easier { background: #eaf6f0; }
98 .verdicts .del { float: right; }
99 .empty { padding: 30px; text-align: center; border: 2px dashed var(--line); border-radius: 14px; color: var(--muted); }
100 .profiles { display: flex; gap: 8px; flex-wrap: wrap; margin: 12px 0; }
101 .chip { padding: 8px 16px; border-radius: 999px; border: 1.5px solid var(--line); background: #fff; cursor: pointer;
font-weight: 700; }
102 .chip.on { background: var(--accent2); color: #fff; border-color: var(--accent2); }
103 table.results { width: 100%; border-collapse: collapse; margin: 16px 0; font-size: 0.95rem; }
104 .results th, .results td { padding: 10px 12px; border-bottom: 1px solid var(--line); text-align: right; }
105 .results th:first-child, .results td:first-child { text-align: left; }
106 .results tr.hl { background: #f1f8f7; font-weight: 700; }
107 .stat { background: var(--soft); border-radius: 14px; padding: 18px; display: flex; flex-direction: column; gap: 4px; }
108 .stat b { font-size: 2.2rem; color: var(--accent); }
109 .stat span { color: #4a4f5c; }
110 .prose { max-width: 820px; margin: 0 auto; line-height: 1.6; }
111 .prose li { margin: 4px 0; }
112 code { background: #f1ece4; padding: 1px 6px; border-radius: 6px; font-size: 0.9em; }
113 .icon { vertical-align: -4px; color: var(--accent); margin-right: 4px; }
114 .brand .icon { width: 26px; height: 26px; vertical-align: -6px; }
```


src/ui/charts.ts

SHA256 c847f206294c4c407f2e01317a397bde30ddc94540327b79ef5750ff0c898759

```
1 import type { Comparison } from "../engine/family";
2 import type { Trial } from "../engine/types";
3
4 const W = 560;
5
6 function esc(s: string): string {
7   return s.replace(/[\&<"']/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" })[c!]);
8 }
9
10 export function posteriorChart(marginal: { srt: number; p: number }[], est?: { lo90: number; hi90: number; srt: number
  }): string {
11   const H = 150;
12   const xMin = marginal[0].srt;
13   const xMax = marginal[marginal.length - 1].srt;
14   const pMax = Math.max(...marginal.map((m) => m.p), 1e-9);
15   const x = (v: number) => 30 + ((v - xMin) / (xMax - xMin)) * (W - 50);
16   const y = (p: number) => H - 25 - (p / pMax) * (H - 45);
17   const path = marginal.map((m, i) => `${i ? "L" : "M"}${x(m.srt).toFixed(1)},${y(m.p).toFixed(1)}`).join("");
18   const area = `${path}L${x(xMax)},${H - 25}L${x(xMin)},${H - 25}Z`;
19   const ticks = [];
20   for (let t = Math.ceil(xMin / 5) * 5; t <= xMax; t += 5)
21     ticks.push(`<line x1=${x(t)}" x2=${x(t)}" y1=${H - 25}" y2=${H - 20}" class="axis"/><text x=${x(t)}" y=${H -
  6}" class="tick">${t}</text>`);
22   const band = est
23     ? `<rect x=${x(est.lo90)}" y="10" width=${Math.max(2, x(est.hi90) - x(est.lo90))}" height=${H - 35}"
  class="band"/><line x1=${x(est.srt)}" x2=${x(est.srt)}" y1="10" y2=${H - 25}" class="mean"/>`
24     : "";
25   return `<svg viewBox="0 0 ${W} ${H}" class="chart" role="img" aria-label="Belief about the model-estimated task
  threshold">
26     ${band}<path d=${area}" class="area"/><path d=${path}" class="line"/>
27     <line x1="30" x2=${W - 20}" y1=${H - 25}" y2=${H - 25}" class="axis"/>${ticks.join("")}
28     <text x=${W - 20}" y=${H - 30}" class="tick" text-anchor="end">SNR (dB) </text>
29   </svg>`;
30 }
31
32 export function trackChart(trials: Trial[]): string {
33   const H = 130;
34   const n = Math.max(12, trials.length);
35   const yMin = -22;
36   const yMax = 10;
37   const x = (i: number) => 44 + (i / (n - 1 || 1)) * (W - 64);
38   const y = (v: number) => 10 + ((yMax - v) / (yMax - yMin)) * (H - 35);
39   const pts = trials.map((t, i) => `<circle cx=${x(i)}" cy=${y(t.snr)}" r="5" class=${t.correct ? "ok" :
  "miss"}"><title>Trial ${i + 1}: ${t.snr} dB, ${t.correct ? "correct" : "missed"}</title></circle>`);
40   const line = trials.map((t, i) => `${i ? "L" : "M"}${x(i)},${y(t.snr)}`).join("");
41   const grid = [-20, -10, 0, 10].map((v) => `<line x1="36" x2=${W - 20}" y1=${y(v)}" y2=${y(v)}" class="grid"/><text
  x="30" y=${y(v) + 4}" class="tick" text-anchor="end">${v}</text>`);
42   return `<svg viewBox="0 0 ${W} ${H}" class="chart" role="img" aria-label="Trial-by-trial SNR
  track">${grid.join("")}<path d=${line}" class="track"/>${pts.join("")}
43   <text x=${W - 20}" y=${H - 6}" class="tick" text-anchor="end">trial </text></svg>`;
44 }
45
46 export function familyChart(rows: Comparison[]): string {
47   if (!rows.length) return "";
48   const rowH = 40;
49   const H = rows.length * rowH + 62;
50   const all = rows.flatMap((r) => [r.member.estimate.lo90, r.member.estimate.hi90]);
51   const xMin = Math.floor(Math.min(...all, -14) / 2) * 2;
52   const xMax = Math.ceil(Math.max(...all, 0) / 2) * 2;
53   const L = 190;
54   const x = (v: number) => L + ((v - xMin) / (xMax - xMin)) * (W - L - 20);
55   const ticks = [];
56   for (let t = xMin; t <= xMax; t += 2) ticks.push(`<line x1=${x(t)}" x2=${x(t)}" y1="10" y2=${H - 48}"
  class="grid"/><text x=${x(t)}" y=${H - 32}" class="tick">${t}</text>`);
57   const items = rows.map((r, i) => {
58     const cy = 14 + i * rowH + rowH / 2;
59     const e = r.member.estimate;
60     return `<text x=${L - 12}" y=${cy + 4}" class="label" text-anchor="end">${esc(r.member.name)}</text>
61     <line x1=${x(e.lo90)}" x2=${x(e.hi90)}" y1=${cy}" y2=${cy}" class="ci ${r.verdict}"/>
62     <circle cx=${x(e.srt)}" cy=${cy}" r="7" class="dot ${r.verdict}><title>${e.srt.toFixed(1)} dB (90% interval
  ${e.lo90.toFixed(1)} to ${e.hi90.toFixed(1)})</title></circle>`;
63   });
64   return `<svg viewBox="0 0 ${W} ${H}" class="chart" role="img" aria-label="Family
  comparison">${ticks.join("")}${items.join("")}
65   <text x=${(L + W - 20) / 2}" y=${H - 10}" class="tick">← copes better in noise · threshold (dB SNR) · needs clearer
  speech </text></svg>`;
66 }
67
```


src/ui/charts.ts (continued)

```
68 export function histogram(values: number[], lo: number, hi: number, bins: number, cls: string): string {
69   const H = 110;
70   const counts = new Array(bins).fill(0);
71   for (const v of values) counts[Math.max(0, Math.min(bins - 1, Math.floor(((v - lo) / (hi - lo)) * bins)))]++;
72   const max = Math.max(...counts, 1);
73   const bw = (W - 50) / bins;
74   const bars = counts.map((c, i) => `
```

src/ui/icons.ts

SHA256 e222748dca7b25e8715629171bbdea3f4b55dc7af18b11cba8a58bef86d007b5

```
1 const svg = (body: string) =>
2   `
```

tests/dsp.test.ts

SHA256 c5e252ea8f1c950a73434fa1db9a7ab8436187390dcc849a649af03f5cd1361f

```
1 import { readFileSync } from "node:fs";
2 import { describe, expect, it } from "vitest";
3 import { dbToGain, fft, rms } from "../src/dsp/fft";
4 import { normalizeRms, speechShapedNoise, trimSilence } from "../src/dsp/stimuli";
5 import { mixTriplet } from "../src/audio/mix";
6 import { lossAt, PROFILES } from "../src/audio/hearloss";
7
8 function readWav(path: string): Float32Array {
9   const b = readFileSync(path);
10   const n = b.readUInt32LE(40) / 2;
11   const out = new Float32Array(n);
12   for (let i = 0; i < n; i++) out[i] = b.readInt16LE(44 + i * 2) / 32768;
13   return out;
14 }
15
16 describe("FFT", () => {
17   it("round-trips", () => {
18     const re = Float64Array.from({ length: 64 }, (_, i) => Math.sin(i) + 0.3 * i);
19     const orig = re.slice();
20     const im = new Float64Array(64);
21     fft(re, im);
22     fft(re, im, true);
23     re.forEach((v, i) => expect(v).toBeCloseTo(orig[i], 9));
24   });
25   it("puts a pure tone in the right bin", () => {
26     const n = 256;
27     const re = Float64Array.from({ length: n }, (_, i) => Math.cos((2 * Math.PI * 10 * i) / n));
28     const im = new Float64Array(n);
29     fft(re, im);
30     const mags = [...re].map((r, i) => Math.hypot(r, im[i]));
31     expect(mags.indexOf(Math.max(...mags.slice(0, n / 2)))).toBe(10);
32   });
33   it("rejects non power-of-two lengths", () => {
34     expect(() => fft(new Float64Array(3), new Float64Array(3))).toThrow();
35   });
36 });
37
38 describe("stimulus processing", () => {
39   it("trims silence and normalizes RMS", () => {
40     const x = new Float32Array(1000);
41     for (let i = 400; i < 600; i++) x[i] = Math.sin(i);
42     const t = trimSilence(x, 1000, -40, 0);
43     expect(t.length).toBeLessThan(220);
44     expect(rms(normalizeRms(t, 0.1))).toBeCloseTo(0.1, 5);
45   });
46   it("makes noise with the speech's spectral tilt", () => {
47     const low = Float32Array.from({ length: 1024 }, (_, i) => Math.sin((2 * Math.PI * 20 * i) / 1024));
48     const noise = speechShapedNoise([low], 4096, 1);
49     expect(noise.length).toBe(4096);
50     expect(rms(noise)).toBeCloseTo(0.1, 4);
51     const re = Float64Array.from(noise);
52     const im = new Float64Array(4096);
53     fft(re, im);
54     const band = (a: number, b: number) => {
55       let s = 0;
56       for (let k = a; k < b; k++) s += re[k] ** 2 + im[k] ** 2;
57       return s;
58     };
59     expect(band(60, 100)).toBeGreaterThan(band(1000, 1040) * 100);
60   });
61 });
62
63 describe("shipped stimuli", () => {
64   it("are RMS-equalized digits and matching noise", () => {
65     for (const d of [0, 1, 2, 3, 4, 5, 6, 8, 9]) {
66       const x = readWav(`public/stimuli/digit-${d}.wav`);
67       expect(x.length / 24000).toBeGreaterThan(0.2);
68       expect(x.length / 24000).toBeLessThan(1);
69       expect(rms(x)).toBeCloseTo(0.1, 2);
70     }
71     expect(rms(readWav("public/stimuli/noise.wav"))).toBeCloseTo(0.1, 2);
72   });
73 });
74
75 describe("triplet mixing", () => {
76   const tok = Float32Array.from({ length: 200 }, (_, i) => 0.1 * Math.sin(i / 3) * Math.SQRT2);
77   const noise = Float32Array.from({ length: 997 }, (_, i) => 0.1 * Math.sin(i * 1.7) * Math.SQRT2);
78   it("sets the requested SNR", () => {
```

tests/dsp.test.ts (continued)

```
79     for (const snr of [-10, 0, 6]) {
80         const m = mixTriplet([tok, tok, tok], new Float32Array(997), { sampleRate: 1000, snr, antiphasic: false, noiseGain:
0.1 });
81         const seg = m.left.slice(Math.round(m.onsets[0] * 1000), Math.round(m.onsets[0] * 1000) + 200);
82         expect(20 * Math.log10(rms(seg) / (0.1 * 0.1))).toBeCloseTo(snr, 1);
83     }
84 });
85 it("inverts speech but not noise in the right ear when antiphasic", () => {
86     const m = mixTriplet([tok, tok, tok], noise, { sampleRate: 1000, snr: 0, antiphasic: true, noiseGain: 0.1 });
87     for (let i = 0; i < 400; i++) expect(m.left[i]).toBeCloseTo(m.right[i], 6);
88     const on = Math.round(m.onsets[1] * 1000) + 50;
89     const sum = m.left[on] + m.right[on];
90     const nz = noise[on % noise.length] * 0.1;
91     expect(sum).toBeCloseTo(2 * nz, 5);
92 });
93 it("never clips", () => {
94     const loud = tok.map((v) => v * 30);
95     const m = mixTriplet([loud, loud, loud], noise, { sampleRate: 1000, snr: 10, antiphasic: true });
96     expect(Math.max(...m.left.map(Math.abs))).toBeLessThanOrEqual(0.95 + 1e-6);
97 });
98 });
99
100 describe("hear-through profiles", () => {
101     it("interpolate in log-frequency and slope downward", () => {
102         const p = PROFILES.find((x) => x.id === "mild-hf");
103         expect(lossAt(p, 250)).toBe(0);
104         expect(lossAt(p, 8000)).toBe(45);
105         expect(lossAt(p, Math.sqrt(2000 * 4000))).toBeCloseTo(27.5);
106         expect(dbToGain(-20)).toBeCloseTo(0.1);
107     });
108 });
```

tests/engine.test.ts

SHA256 d2283865de78146b208175de9453338541f28bd1fba8d196510a3b4177afae66

```
1 import { describe, expect, it } from "vitest";
2 import { BayesianDin } from "../src/engine/bayes";
3 import { Staircase } from "../src/engine/staircase";
4 import { pCorrect } from "../src/engine/psychometric";
5 import { mulberry32 } from "../src/engine/rng";
6 import { DEFAULT_SIM, METHODS, runOnce, simulate } from "../src/engine/simulate";
7 import { DIGITS, makeTriplet, parseResponse, scoreTriplet } from "../src/engine/triplets";
8 import { headphonePassed, makeHeadphoneTrial } from "../src/engine/headphones";
9
10 describe("psychometric function", () => {
11   it("is 50% of (1 - lapse) at the SRT and monotonic", () => {
12     expect(pCorrect(-6, -6, 0.7, 0)).toBeCloseTo(0.5);
13     expect(pCorrect(-10, -6)).toBeLessThan(pCorrect(-6, -6));
14     expect(pCorrect(10, -6, 0.7, 0.03)).toBeLessThanOrEqual(0.97);
15   });
16 });
17
18 describe("triplets", () => {
19   it("uses three distinct digits from the set and never 7", () => {
20     const rng = mulberry32(1);
21     for (let i = 0; i < 500; i++) {
22       const t = makeTriplet(rng);
23       expect(new Set(t).size).toBe(3);
24       for (const d of t) expect(DIGITS).toContain(d);
25       expect(t).not.toContain(7);
26     }
27   });
28   it("scores whole-triplet correctness in order", () => {
29     expect(scoreTriplet([1, 2, 3], "123")).toEqual({ correct: true, digitsCorrect: 3 });
30     expect(scoreTriplet([1, 2, 3], "1 2 4")).toEqual({ correct: false, digitsCorrect: 2 });
31     expect(scoreTriplet([1, 2, 3], "321").correct).toBe(false);
32     expect(scoreTriplet([1, 2, 3], "1234").correct).toBe(false);
33     expect(parseResponse("a1-2 3")).toEqual([1, 2, 3]);
34   });
35 });
36
37 describe("staircase", () => {
38   it("moves down 2 dB after a correct triplet and up after a miss", () => {
39     const s = new Staircase();
40     expect(s.nextSnr()).toBe(0);
41     s.update({ snr: 0, correct: true });
42     expect(s.nextSnr()).toBe(-2);
43     s.update({ snr: -2, correct: false });
44     expect(s.nextSnr()).toBe(0);
45   });
46   it("runs exactly 24 trials", () => {
47     const e = runOnce(METHODS.staircase, -8, 0.7, 0.02, mulberry32(3));
48     expect(e.trials).toBe(24);
49   });
50 });
51
52 describe("Bayesian procedure", () => {
53   it("starts with a broad belief and narrows after informative answers", () => {
54     const b = new BayesianDin();
55     const sd0 = b.estimate().sd;
56     const rng = mulberry32(5);
57     for (let i = 0; i < 10; i++) {
58       const snr = b.nextSnr();
59       b.update({ snr, correct: rng() < pCorrect(snr, -9) });
60     }
61     expect(b.estimate().sd).toBeLessThan(sd0 / 2);
62     const total = b.marginal().reduce((a, m) => a + m.p, 0);
63     expect(total).toBeCloseTo(1, 6);
64   });
65   it("places trials near the current belief, not at the extremes", () => {
66     const b = new BayesianDin();
67     const first = b.nextSnr();
68     expect(first).toBeGreaterThan(-16);
69     expect(first).toBeLessThan(0);
70   });
71   it("moves its estimate toward the listener", () => {
72     const good = runOnce(METHODS.bayesian, -12, 0.7, 0.02, mulberry32(11));
73     const poor = runOnce(METHODS.bayesian, 0, 0.7, 0.02, mulberry32(11));
74     expect(good.srt).toBeLessThan(-9);
75     expect(poor.srt).toBeGreaterThan(-3);
76   });
77   it("respects min/max trial limits", () => {
78     const e = runOnce(METHODS.bayesian, -6, 0.7, 0.02, mulberry32(2));
```

tests/engine.test.ts (continued)

```
79     expect(e.trials).toBeGreaterThanOrEqual(10);
80     expect(e.trials).toBeLessThanOrEqual(24);
81   });
82 });
83
84 describe("simulation study (the headline claims)", () => {
85   const cfg = { ...DEFAULT_SIM, listeners: 200 };
86   const s24 = simulate(METHODS.staircase, cfg);
87   const s12 = simulate(METHODS.staircase12, cfg);
88   const b = simulate(METHODS.bayesian, cfg);
89
90   it("is deterministic for a given seed", () => {
91     expect(simulate(METHODS.bayesian, { ...cfg, listeners: 30 }).rmse).toBe(simulate(METHODS.bayesian, { ...cfg,
listeners: 30 }).rmse);
92   });
93   it("beats a staircase of the same length on error", () => {
94     expect(b.meanTrials).toBeLessThan(14);
95     expect(b.rmse).toBeLessThan(s12.rmse * 0.8);
96   });
97   it("uses roughly half the trials of the classic 24-trial track", () => {
98     expect(b.meanTrials).toBeLessThan(s24.meanTrials * 0.6);
99   });
100   it("reports 90% intervals that are approximately calibrated", () => {
101     expect(b.coverage90).toBeGreaterThan(0.85);
102     expect(b.coverage90).toBeGreaterThan(s24.coverage90);
103   });
104 });
105
106 describe("headphone check", () => {
107   it("never makes the quiet tone the antiphase tone", () => {
108     const rng = mulberry32(9);
109     for (let i = 0; i < 200; i++) {
110       const t = makeHeadphoneTrial(rng);
111       expect(t.quiet).not.toBe(t.antiphase);
112       expect([0, 1, 2]).toContain(t.quiet);
113       expect([0, 1, 2]).toContain(t.antiphase);
114     }
115   });
116   it("passes at 5 of 6", () => {
117     expect(headphonePassed(5)).toBe(true);
118     expect(headphonePassed(4)).toBe(false);
119   });
120 });
```

tests/family.test.ts

SHA256 b1315dcacde58452a9b753f3feac01e8816a102d9a02e692d0f5c76568b27b78

```
1 import { describe, expect, it } from "vitest";
2 import { compareGroup, median, type Member } from "../src/engine/family";
3 import { demoFamily } from "../src/storage";
4
5 const mk = (id: string, srt: number, sd = 1): Member => ({
6   id,
7   name: id,
8   estimate: { srt, sd, lo90: srt - 1.645 * sd, hi90: srt + 1.645 * sd, trials: 12 },
9   takenAt: "2026-01-01T00:00:00Z",
10  device: "d",
11 });
12
13 describe("family comparison", () => {
14   it("computes medians", () => {
15     expect(median([3, 1, 2])).toBe(2);
16     expect(median([4, 1, 2, 3])).toBe(2.5);
17   });
18   it("needs at least two people", () => {
19     expect(compareGroup([mk("a", -10)])[0].verdict).toBe("too-few");
20   });
21   it("flags only the clear outlier", () => {
22     const rows = compareGroup([mk("a", -11), mk("b", -10), mk("c", -9.5), mk("grandpa", -3)]);
23     expect(rows.map((r) => r.verdict)).toEqual(["similar", "similar", "similar", "harder"]);
24     expect(rows[3].delta).toBeCloseTo(7);
25   });
26   it("does not flag small gaps even with tight intervals", () => {
27     const rows = compareGroup([mk("a", -10, 0.2), mk("b", -8, 0.2)]);
28     expect(rows.every((r) => r.verdict === "similar")).toBe(true);
29   });
30   it("does not flag larger gaps when both measurements are too uncertain", () => {
31     const rows = compareGroup([mk("a", -10, 3), mk("b", -5, 3)]);
32     expect(rows.every((r) => r.verdict === "similar")).toBe(true);
33   });
34   it("is invariant to a shared device offset", () => {
35     const base = [mk("a", -11), mk("b", -10), mk("c", -3)];
36     const shifted = base.map((m) => mk(m.id, m.estimate.srt + 6.5));
37     expect(compareGroup(shifted).map((r) => r.verdict)).toEqual(compareGroup(base).map((r) => r.verdict));
38   });
39   it("flags Grandpa in the synthetic demo family", () => {
40     const rows = compareGroup(demoFamily());
41     expect(rows.filter((r) => r.verdict === "harder").map((r) => r.member.name)).toEqual(["Grandpa (synthetic)"]);
42   });
43 });
```

tsconfig.json

SHA256 53f4f03399bb34af16ef7ca8bc8ae680cc27926cecb41a9f508f8c075e9ac818

```
1 {
2   "compilerOptions": {
3     "target": "ES2022",
4     "module": "ESNext",
5     "moduleResolution": "bundler",
6     "lib": ["ES2022", "DOM", "DOM.Iterable", "WebWorker"],
7     "strict": true,
8     "noUnusedLocals": true,
9     "noUnusedParameters": true,
10    "noImplicitReturns": true,
11    "noFallthroughCasesInSwitch": true,
12    "isolatedModules": true,
13    "skipLibCheck": true,
14    "types": ["vite/client", "node"]
15  },
16  "include": ["src", "tests", "scripts", "vite.config.ts"]
17 }
```


vite.config.ts

SHA256 747168ba945a3d64238e62f9e022fec96ac2ea3060158c52762c5f4da3706cfe

```
1 import { defineConfig } from "vitest/config";
2
3 export default defineConfig({
4   base: "./",
5   build: {
6     target: "es2022",
7   },
8   test: {
9     environment: "node",
10    include: ["tests/**/*.test.ts"],
11    testTimeout: 120_000,
12  },
13 });
```